

(12) **United States Patent**
Chen et al.

(10) **Patent No.:** **US 12,407,839 B2**
(45) **Date of Patent:** ***Sep. 2, 2025**

(54) **SIMPLIFICATIONS OF CROSS-COMPONENT LINEAR MODEL**

(71) Applicant: **BEIJING DAJIA INTERNET INFORMATION TECHNOLOGY CO., LTD.**, Beijing (CN)

(72) Inventors: **Yi-wen Chen**, San Diego, CA (US);
Xianglin Wang, San Diego, CA (US)

(73) Assignee: **BEIJING DAJIA INTERNET INFORMATION TECHNOLOGY CO., LTD.**, Beijing (CN)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 0 days.

This patent is subject to a terminal disclaimer.

(21) Appl. No.: **18/750,699**

(22) Filed: **Jun. 21, 2024**

(65) **Prior Publication Data**
US 2024/0348802 A1 Oct. 17, 2024

Related U.S. Application Data
(63) Continuation of application No. 18/212,640, filed on Jun. 21, 2023, now Pat. No. 12,063,377, which is a (Continued)

(51) **Int. Cl.**
H04N 19/186 (2014.01)
H04N 19/105 (2014.01)
(Continued)

(52) **U.S. Cl.**
CPC **H04N 19/186** (2014.11); **H04N 19/105** (2014.11); **H04N 19/132** (2014.11);
(Continued)

(58) **Field of Classification Search**
CPC H04N 19/105; H04N 19/11; H04N 19/119; H04N 19/132; H04N 19/157;
(Continued)

(56) **References Cited**

U.S. PATENT DOCUMENTS

11,323,726 B2 * 5/2022 Chen H04N 19/59
11,632,559 B2 * 4/2023 Chen H04N 19/593
375/240.25

(Continued)

FOREIGN PATENT DOCUMENTS

CN 103918269 A 7/2014
CN 104380741 A 2/2015

(Continued)

OTHER PUBLICATIONS

“Enhanced Cross-Component Linear Model for Chroma Intra-Prediction in Video Coding”—Kai Zhang, Jianle Chen, Li Zhang, Xiang Li, Marta Karczewicz; IEEE Transactions on Image Processing, vol. 27, No. 8, Aug. 2018. (Year: 2018)*

(Continued)

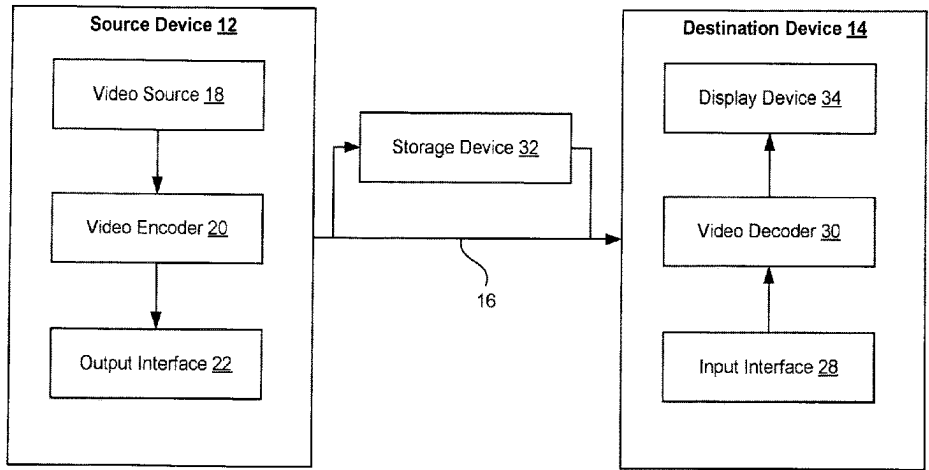
Primary Examiner — Mainul Hasan

(74) Attorney, Agent, or Firm — Scully, Scott, Murphy & Presser, P.C.

(57) **ABSTRACT**

A computing device performs a method of decoding video data by reconstructing a luma block corresponding to a chroma block; searching a sub-group of a plurality of reconstructed neighboring luma samples in a predefined order to identify a maximum luma sample and a minimum luma sample; computing a down-sampled maximum luma sample corresponding to the maximum luma sample; computing a down-sampled minimum luma sample corresponding to the minimum luma sample; generating a linear model using the down-sampled maximum luma sample, the down-sampled minimum luma sample, the first reconstructed chroma sample, and the second reconstructed chroma sample; computing down-sampled luma samples from luma samples of the reconstructed luma block, wherein each down-sampled luma sample corresponds to a chroma sample of the chroma block; and predicting chroma samples of the

(Continued)



chroma block by applying the liner model to the corresponding down-sampled luma samples.

15 Claims, 15 Drawing Sheets

Related U.S. Application Data

continuation of application No. 18/126,179, filed on Mar. 24, 2023, now Pat. No. 11,962,789, which is a continuation of application No. 17/700,238, filed on Mar. 21, 2022, now Pat. No. 11,632,559, which is a continuation of application No. 17/225,955, filed on Apr. 8, 2021, now Pat. No. 11,323,726, which is a continuation of application No. PCT/US2019/055208, filed on Oct. 8, 2019.

- (60) Provisional application No. 62/742,806, filed on Oct. 8, 2018.
- (51) **Int. Cl.**
H04N 19/132 (2014.01)
H04N 19/176 (2014.01)
H04N 19/30 (2014.01)
H04N 19/44 (2014.01)
H04N 19/59 (2014.01)
- (52) **U.S. Cl.**
 CPC **H04N 19/176** (2014.11); **H04N 19/30** (2014.11); **H04N 19/44** (2014.11); **H04N 19/59** (2014.11)
- (58) **Field of Classification Search**
 CPC H04N 19/186; H04N 19/44; H04N 19/59;
 H04N 19/593
 See application file for complete search history.

(56) References Cited

U.S. PATENT DOCUMENTS

11,962,789 B2 *	4/2024	Chen	H04N 19/593
12,063,377 B2 *	8/2024	Chen	H04N 19/105
2012/0328013 A1	12/2012	Budagavi et al.	
2013/0188703 A1 *	7/2013	Liu	H04N 19/50 375/240.12
2013/0322746 A1	12/2013	Cote et al.	
2015/0124865 A1	5/2015	Kim et al.	
2016/0277762 A1 *	9/2016	Zhang	H04N 19/593
2017/0150183 A1	5/2017	Zhang et al.	
2017/0295365 A1 *	10/2017	Budagavi	H04N 19/167
2017/0359595 A1	12/2017	Zhang et al.	
2017/0366818 A1	12/2017	Zhang et al.	
2017/0374372 A1	12/2017	Liu et al.	
2018/0048889 A1	2/2018	Zhang et al.	
2018/0063527 A1	3/2018	Chen et al.	
2018/0070107 A1	3/2018	Ramasubramonian et al.	
2018/0077426 A1 *	3/2018	Zhang	H04N 19/157
2018/0152721 A1	5/2018	Rusanovskyy et al.	
2018/0205946 A1	7/2018	Zhang et al.	
2019/0045184 A1	2/2019	Zhang et al.	
2020/0195930 A1 *	6/2020	Choi	H04N 19/159
2020/0288135 A1	9/2020	Laroche et al.	
2020/0413062 A1 *	12/2020	Onno	H04N 19/186
2020/0413069 A1	12/2020	Lim et al.	
2021/0136392 A1 *	5/2021	Ma	H04N 19/186
2021/0136409 A1 *	5/2021	Ma	H04N 19/132
2021/0227235 A1	7/2021	Chen et al.	

FOREIGN PATENT DOCUMENTS

CN	104471940 A	3/2015
CN	105723707 A	6/2016

CN	107302699 A	10/2017
CN	107371040 A	11/2017
CN	107431795 A	12/2017
CN	108604391 A	9/2018
EP	3272120 A1	1/2018
KR	20120140366 A	12/2012
WO	2008145039 A1	12/2008
WO	2017140211 A1	8/2017
WO	2017203930 A1	11/2017
WO	2017214420 A1	12/2017
WO	2018053293 A1	3/2018
WO	2018118940 A1	6/2018
WO	2020263392 A1	12/2020

OTHER PUBLICATIONS

“Chroma Intra Prediction Based on Inter-Channel Correlation for HEVC”—Xingyu Zhang, Christophe Gisquet, Edouard François, Feng Zou, Oscar C. Au; IEEE Transactions on Image Processing, vol. 23, No. 1, Jan. 2014. (Year: 2014)*

International Search Report and Written Opinion dated Feb. 3, 2020 received in International Application No. PCT/US2019/055208.

International Preliminary Report on Patentability dated Apr. 8, 2021 received in International Application No. PCT/US2019/055208.

Office Action dated Jun. 4, 2021 received in U.S. Appl. No. 17/225,955.

Office Action dated Sep. 24, 2021 received in U.S. Appl. No. 17/225,955.

Notice of Allowance dated Dec. 29, 2021 received in U.S. Appl. No. 17/225,955.

Jianle Chen et al. Algorithm Description of Joint Exploration Test Model 4, JVET of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, Document JVET-D1001 v1, 4th Meeting, Chengdu, CN, Oct. 15-21, 2016, 38 pgs.

Jianle Chen et al. Algorithm Description of Joint Exploration Test Model 5, Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, Document JVET-E1001 v1, 5th Meeting: Geneva, CH, Jan. 12-20, 2017, 44 pgs.

Kai Zhang, et al., CE3-related: CCLM prediction with single-line neighbouring luma samples, Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC1/SC 29/WG 11, JVET-L329-v1, 12th Meeting: Macao, CN, Oct. 3-12, 2018, pp. 1-3, 5 pgs.

Philippe Hanhart, et al., CE3-related: Enabling different chroma sample location types in CCLM, Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, JVET-L0239-v2, 12th Meeting: Macau, CN, Oct. 3-12, 2018, pp. 1-7, 10 pgs.

Xiang Ma, CE3: CCLM/MDLM using simplified coefficients derivation method (Test 5.6.1, 5.6.2 and 5.6.3), Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, Document: JVET-L0340_rl, 12th Meeting: Macao, CN, Oct. 3-12, 2018, 6 pgs.

“Enhanced Cross-Component Linear Model for Chroma Intra Prediction in Video Coding”—Kai Zhang, Jianle Chen, Li Zhang, Xiang Li, Marta Karczewicz; IEEE Transactions on Image Processing, vol. 27, No. 8, Aug. 2018.

“Chroma Intra Prediction Based on Inter-Channel Correlation for HEVC”—Xingyu ZHANG, Christophe Gisquet, Edouard François, Feng Zou, Oscar C. Au; IEEE Transactions on Image Processing vol. 23 No. 1, Jan. 2014.

Office Action dated Nov. 9, 2022 received in U.S. Appl. No. 17/700,238.

Office Action dated Nov. 23, 2021 received in U.S. Appl. No. 17/222,461.

First Office Action of CN202210198549.0 from CNIPA dated Jul. 13, 2022.

Laroche et al.; “Non-CE3: On cross-component linear model simplification”, Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, Document: JVET-K0204-v3, 11th Meeting: Ljubljana, SI, Jul. 10-18, 2018, 7 pgs.

Notice of Allowance dated Feb. 7, 2023 received in U.S. Appl. No. 17/700,238.

(56)

References Cited

OTHER PUBLICATIONS

First Office Action dated Jan. 12, 2024 received in Chinese Patent Application No. CN 202310987345.X.

First Office Action dated Dec. 4, 2023 received in Chinese Patent Application No. CN 202310744433.7.

Office Action dated Dec. 19, 2023 received in Japanese Patent Application No. JP 2023-040322.

Yan, X. et al., "Bitrate decrease by interleave intra prediction coding for HEVC", Video engineering, 2017, 41(9/10), pp. 27-32.

Notification of Due Registration Formalities and Granting Patent Right to Invention dated Feb. 1, 2024 received in Chinese Patent Application No. CN 202310242277.4.

Office Action dated Oct. 24, 2023 received in U.S. Appl. No. 18/126,179.

Xiang Ma, et al., CE3: Classification-based mean value for CCLM coefficients derivation (tests 2.5.1-2.5.4). Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, JVET-M0401-v1.13 th Meeting: Marrakech, MA, 2019年 01月, pp. 1-5.

Gang-Yi, J. et al., "A Novel Fast Mode Selection Algorithm for Intra Prediction Based on Characteristics of Block's Reference Pixels", Journal of Electronics & Information Technology, Oct. 2006, pp. 1874-1878, vol. 28, No. 10.

First Office Action dated Feb. 5, 2024 received in Chinese Patent Application No. CN 202311000873.8.

Notification of Due Registration Formalities and Granting Patent Right to Invention dated Feb. 4, 2024 received in Chinese Patent Application No. CN 202311006402.8.

Notification of Due Registration Formalities and Granting Patent Right to Invention dated Feb. 1, 2024 received in Chinese Patent Application No. CN 202310744433.7.

* cited by examiner

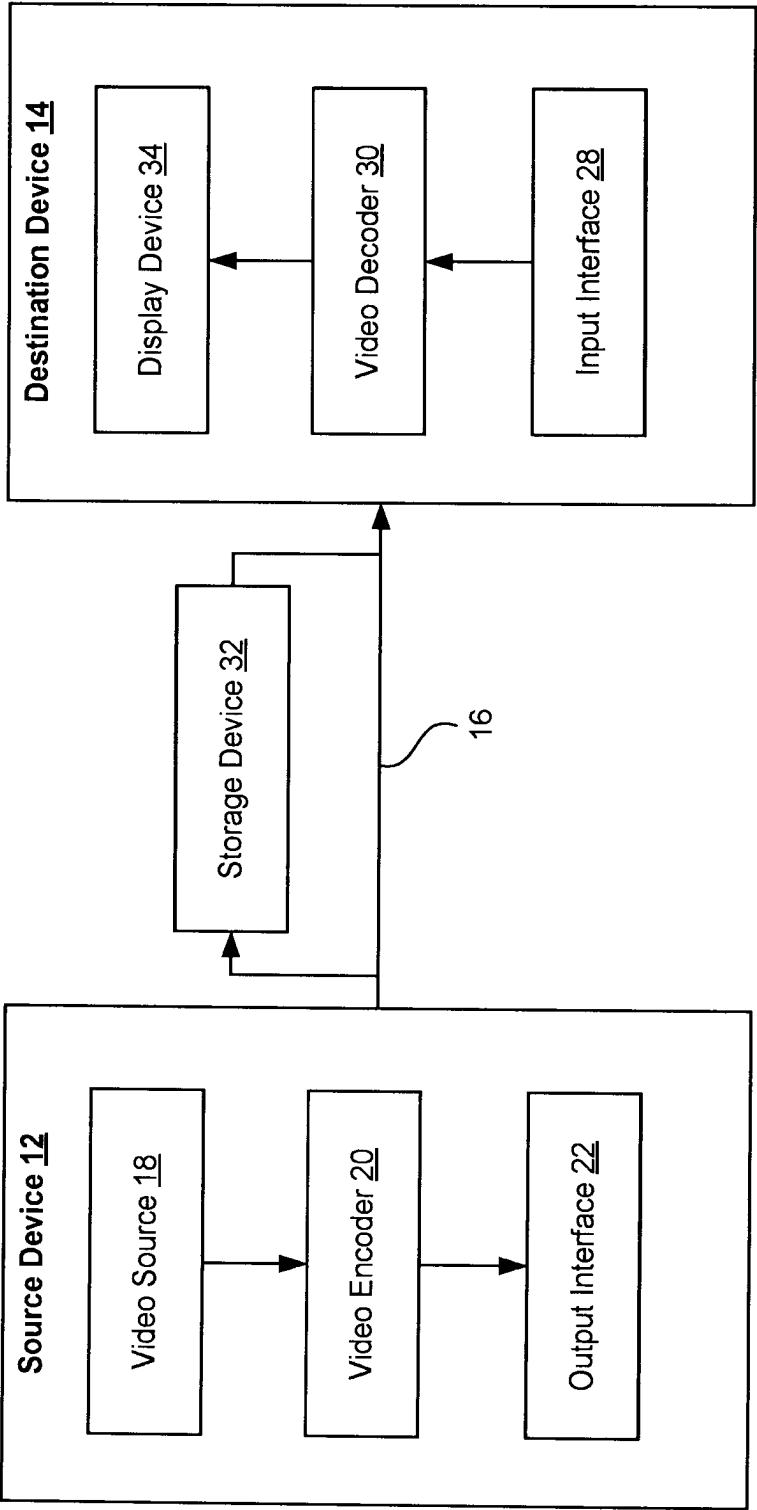


FIG. 1

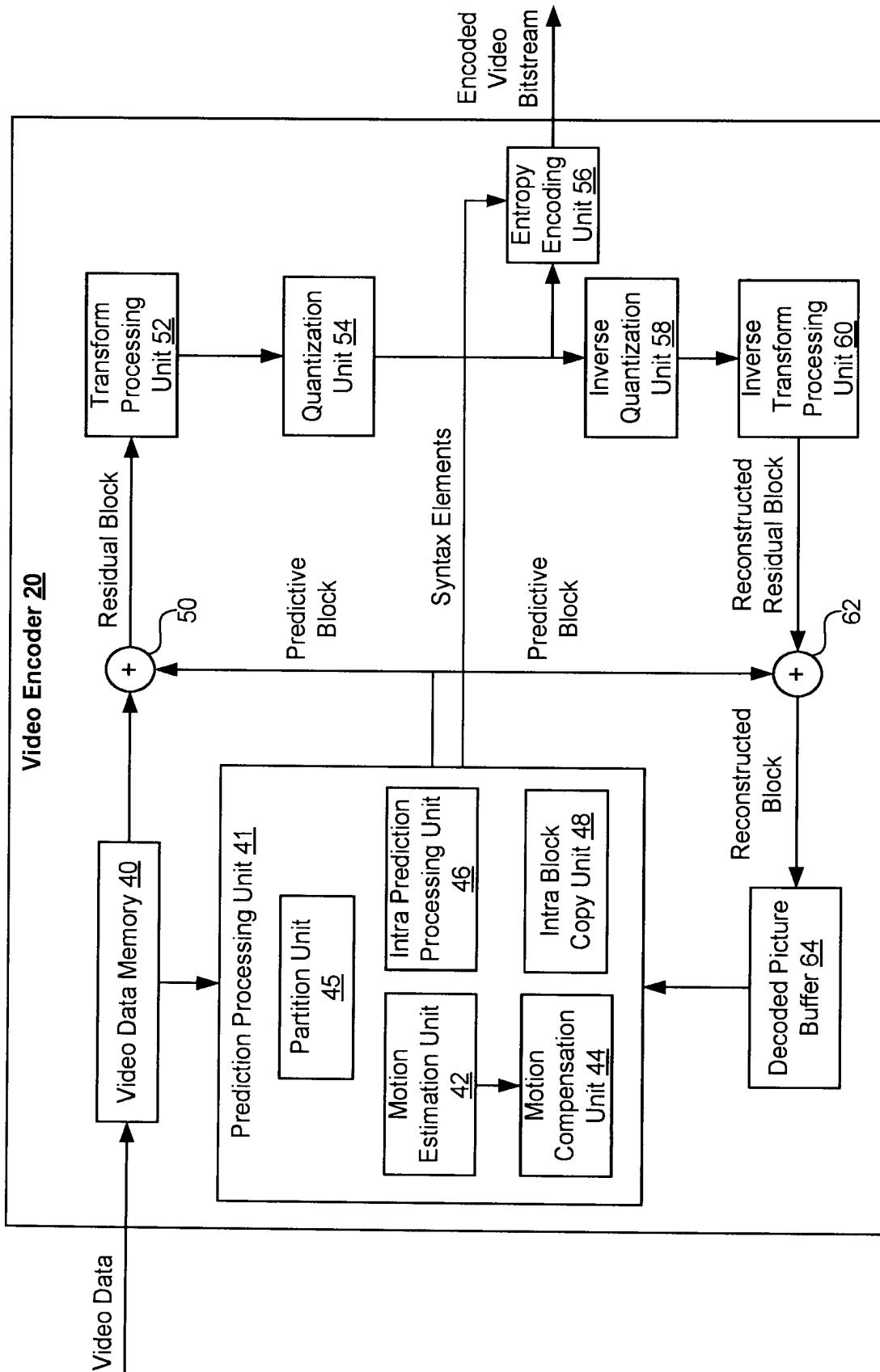


FIG. 2

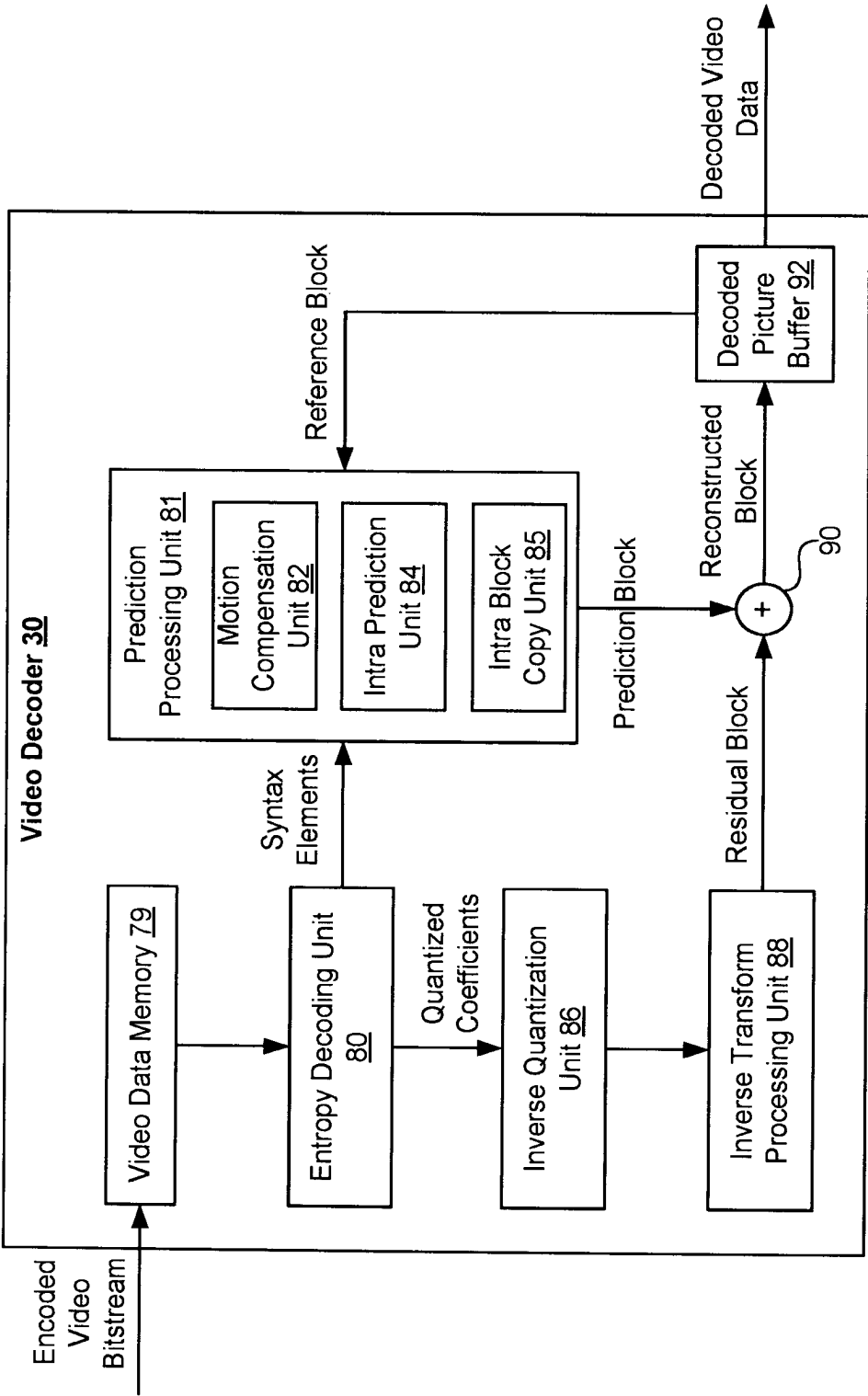


FIG. 3

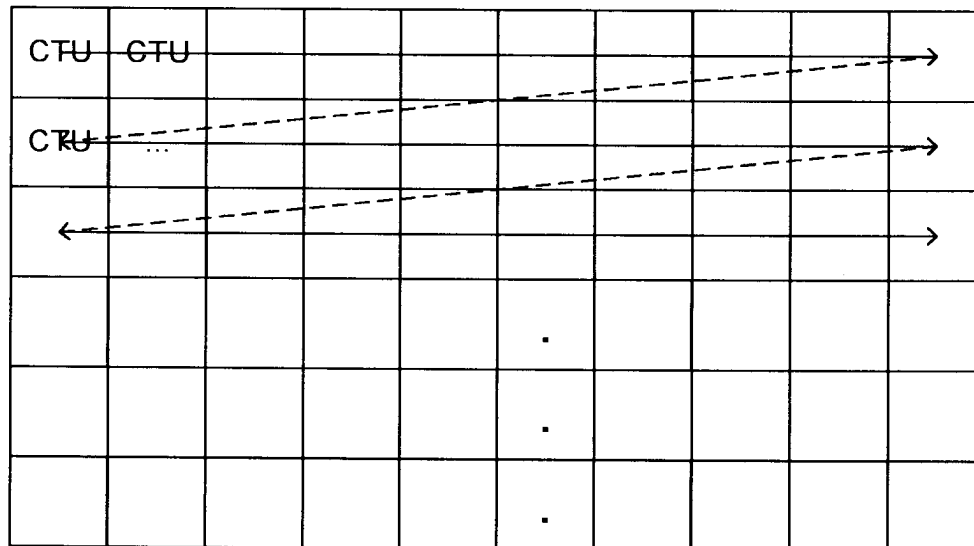


FIG. 4A

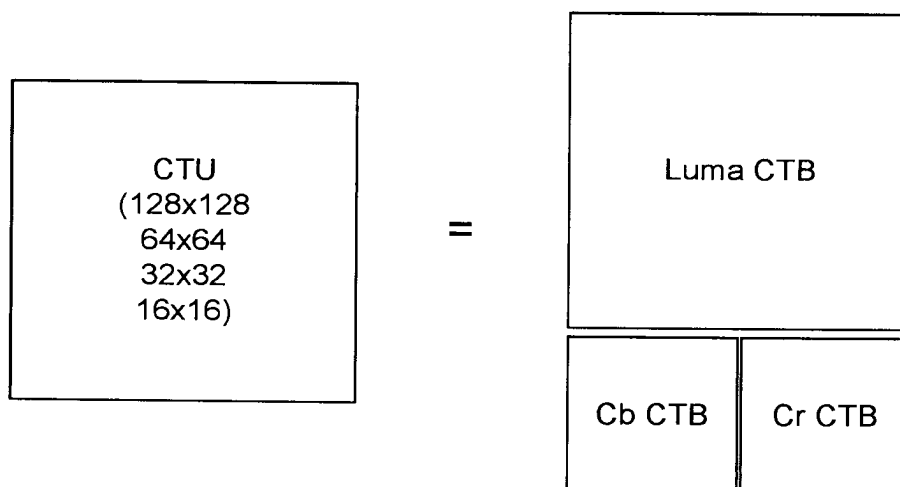


FIG. 4B

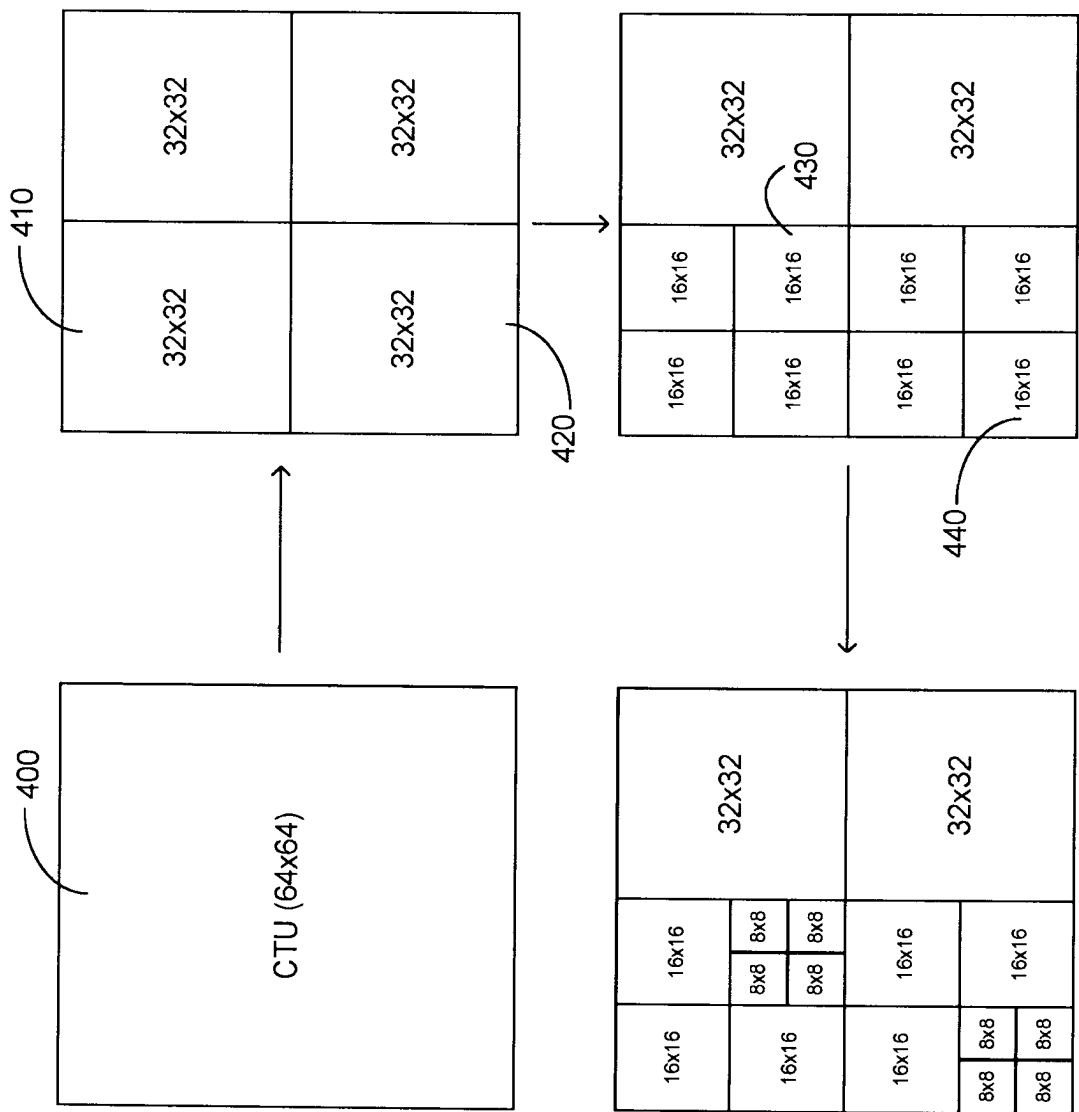


FIG. 4C

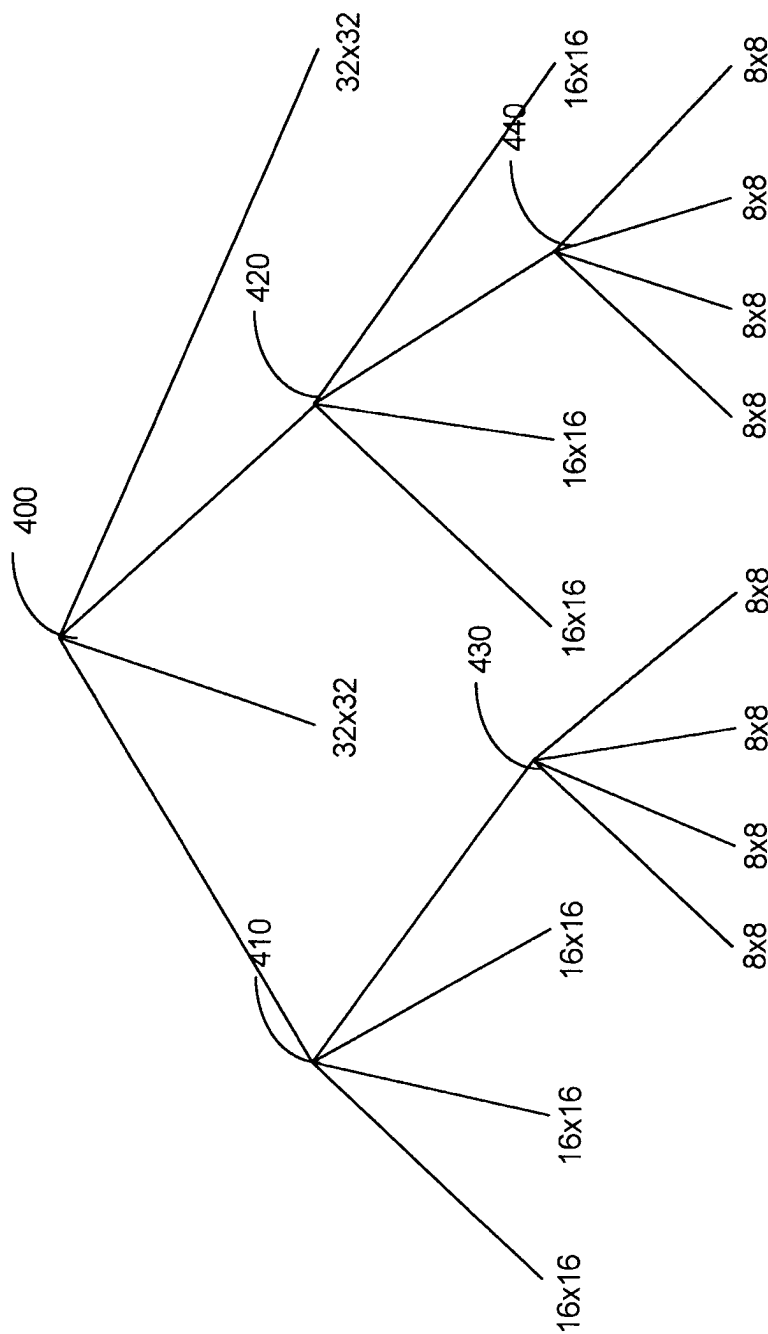


FIG. 4D

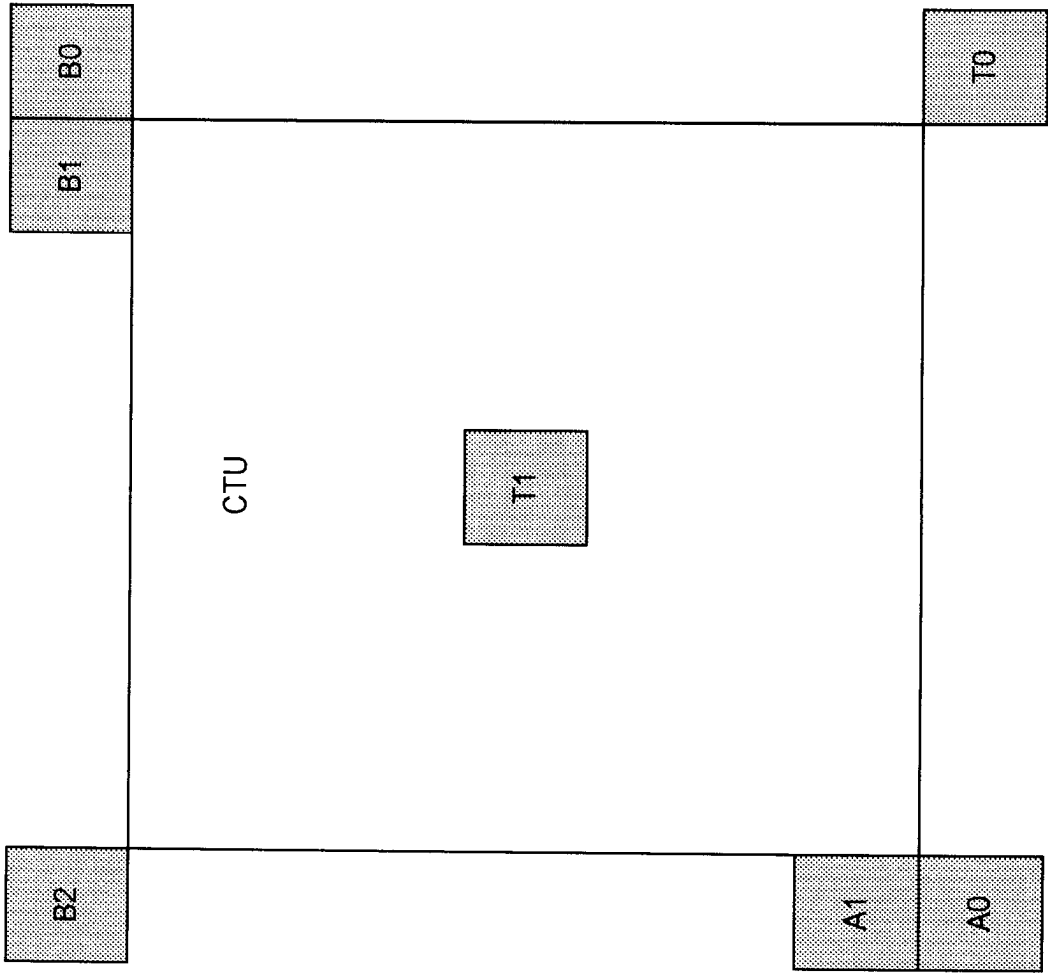


FIG. 5A

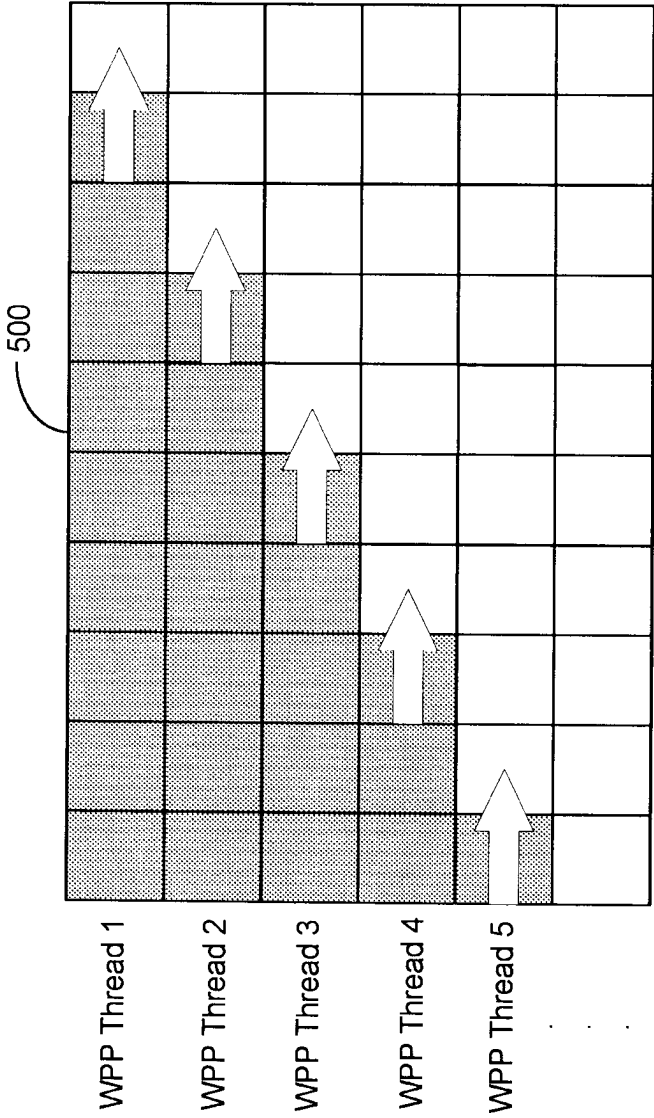
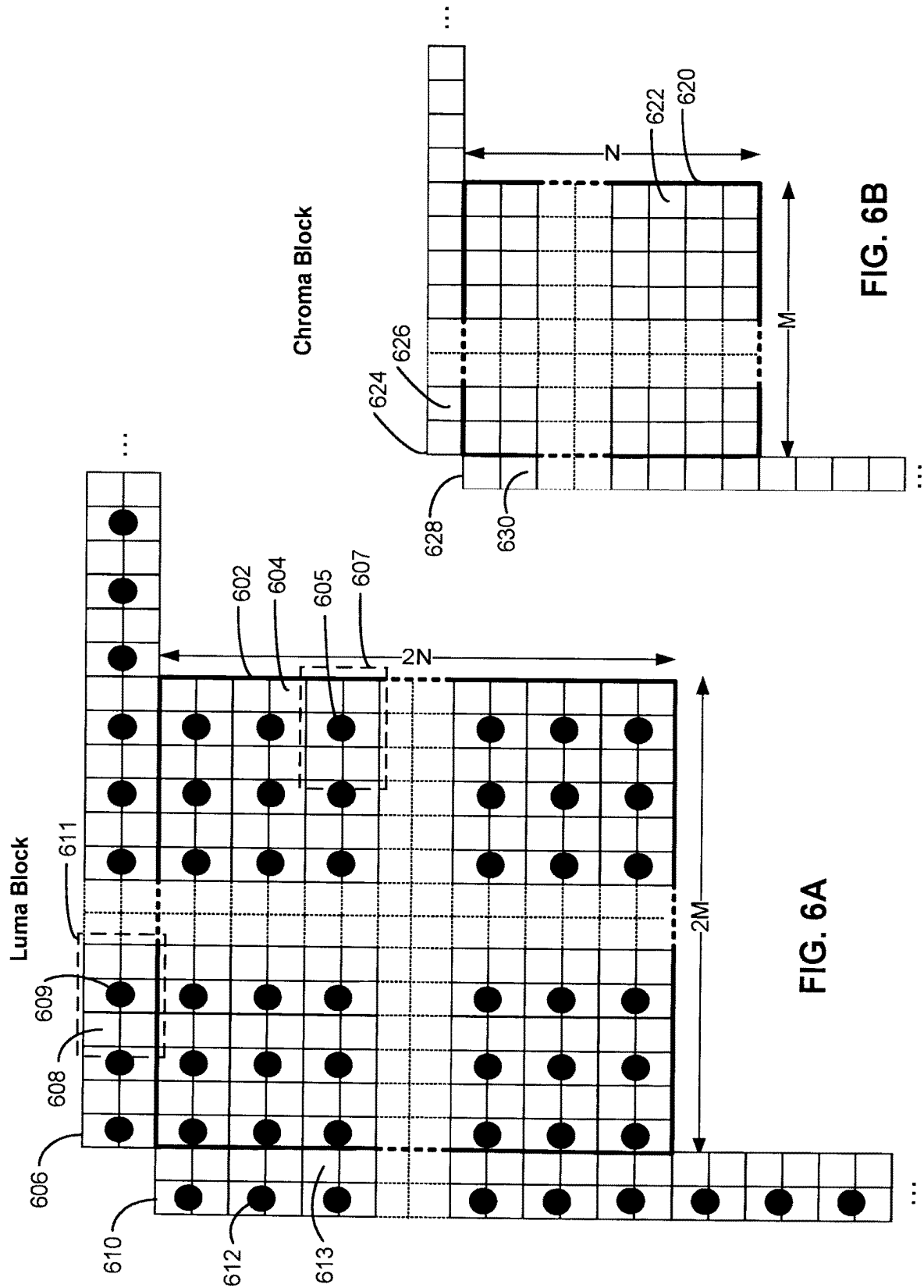


FIG. 5B



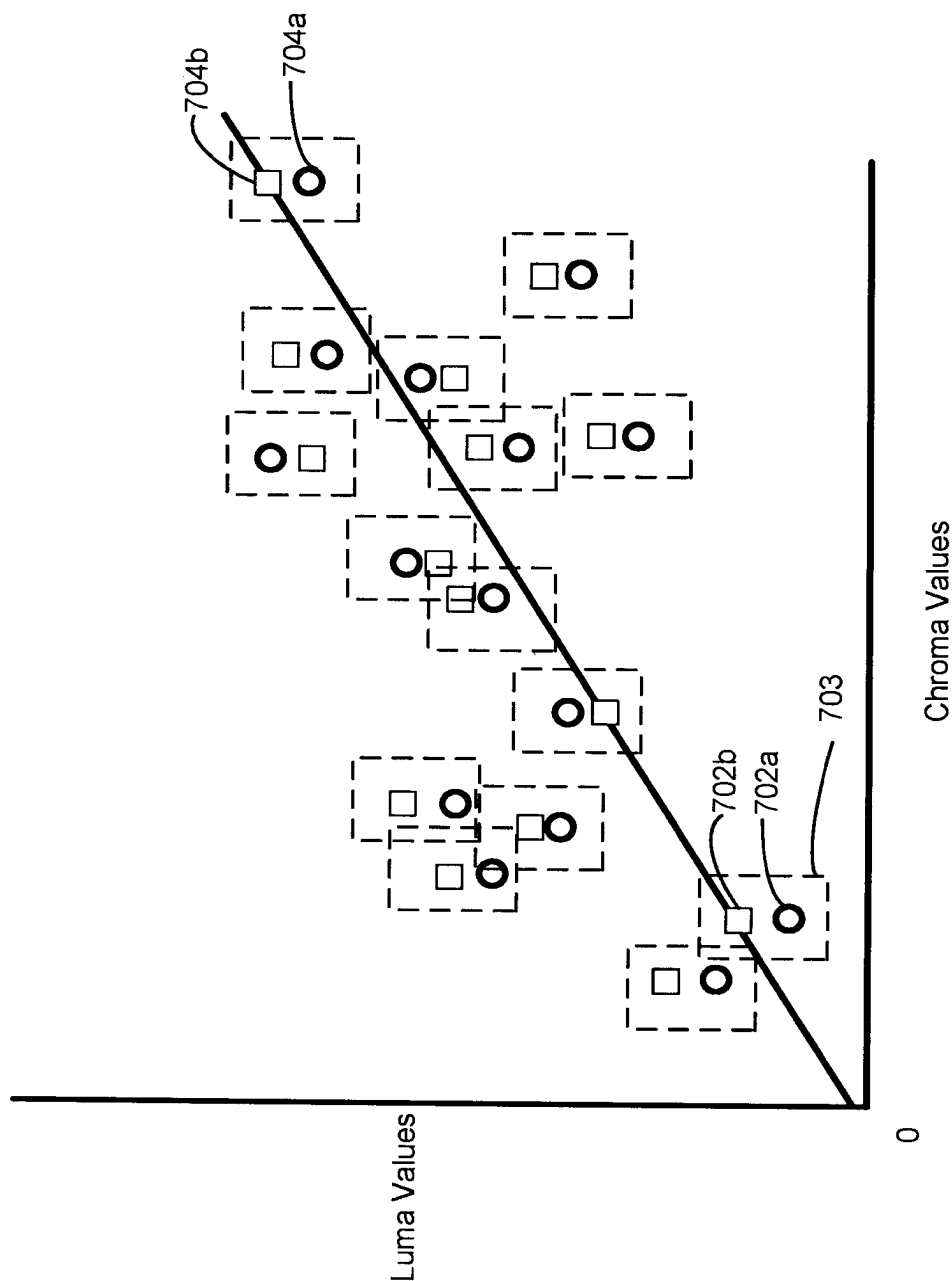


FIG. 7A

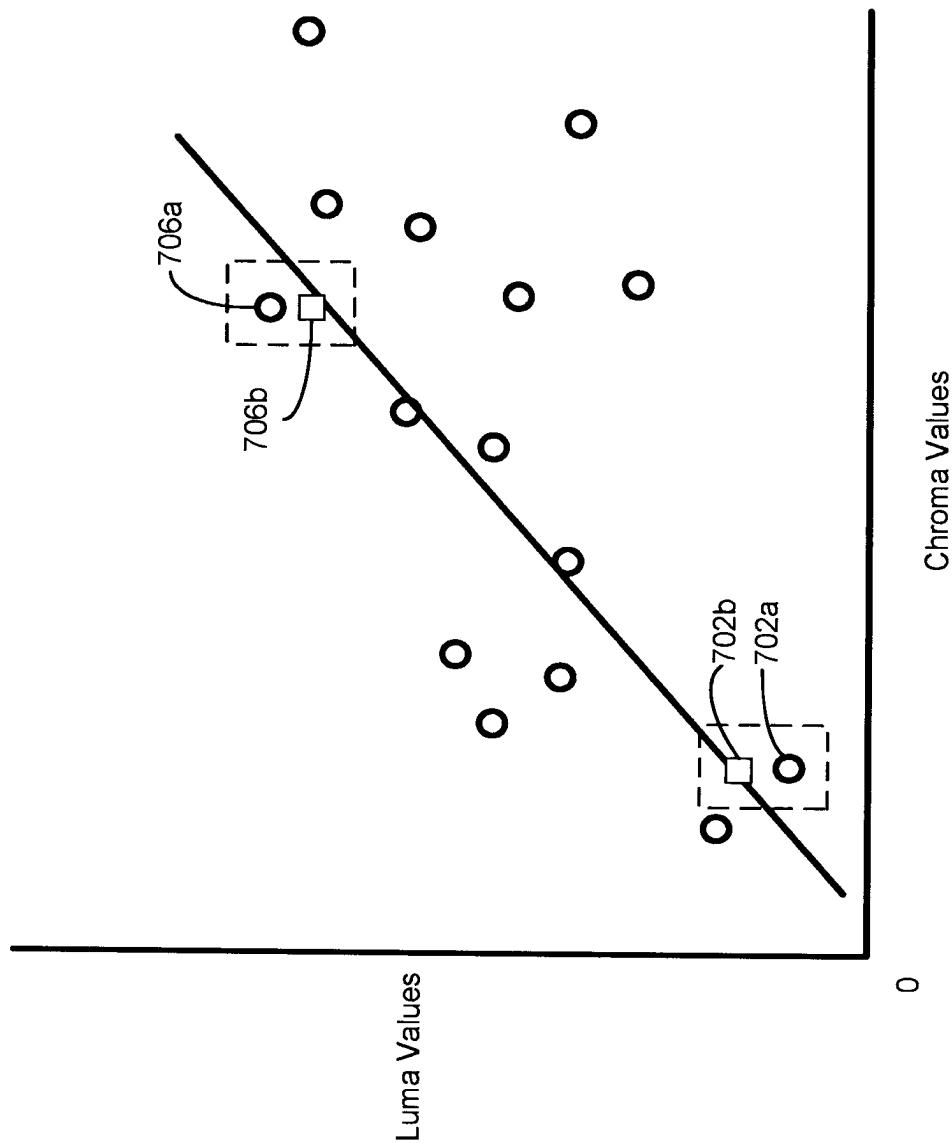


FIG. 7B

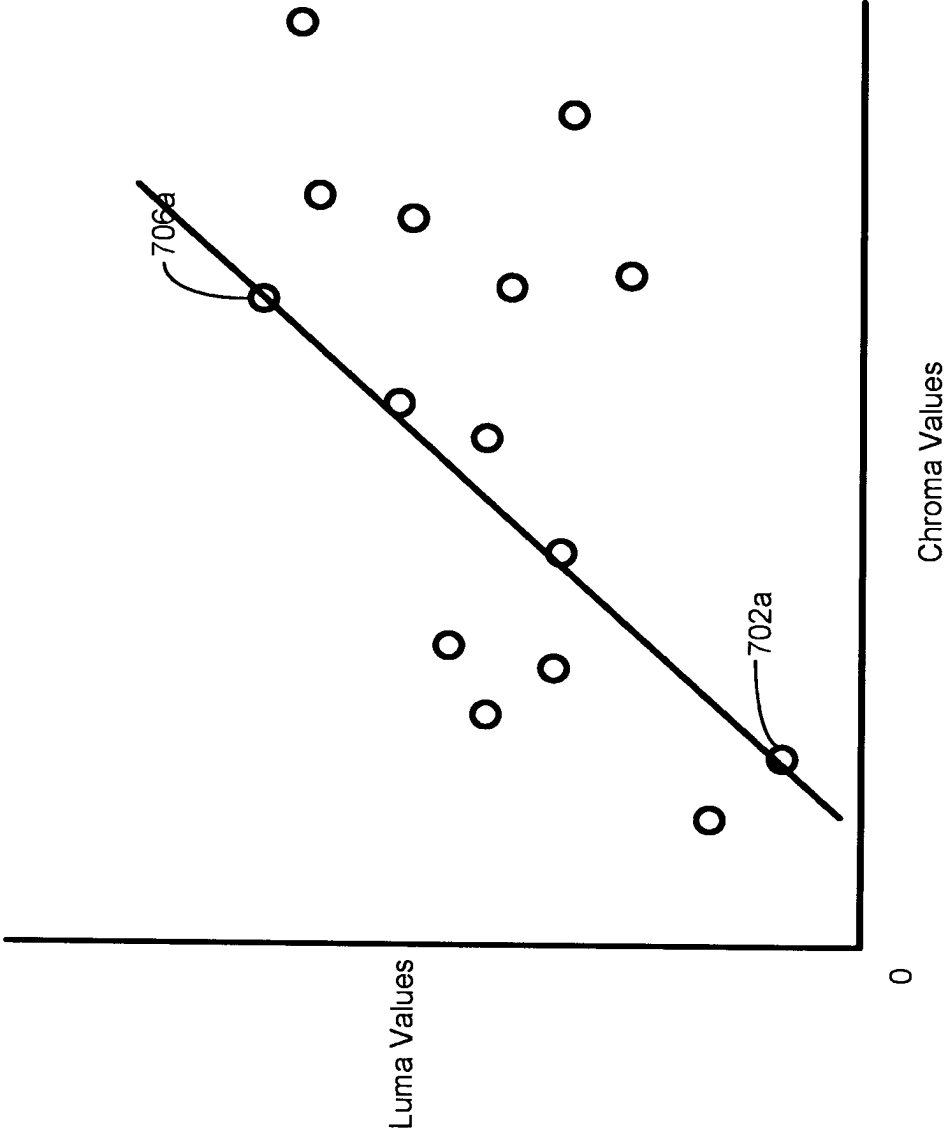


FIG. 7C

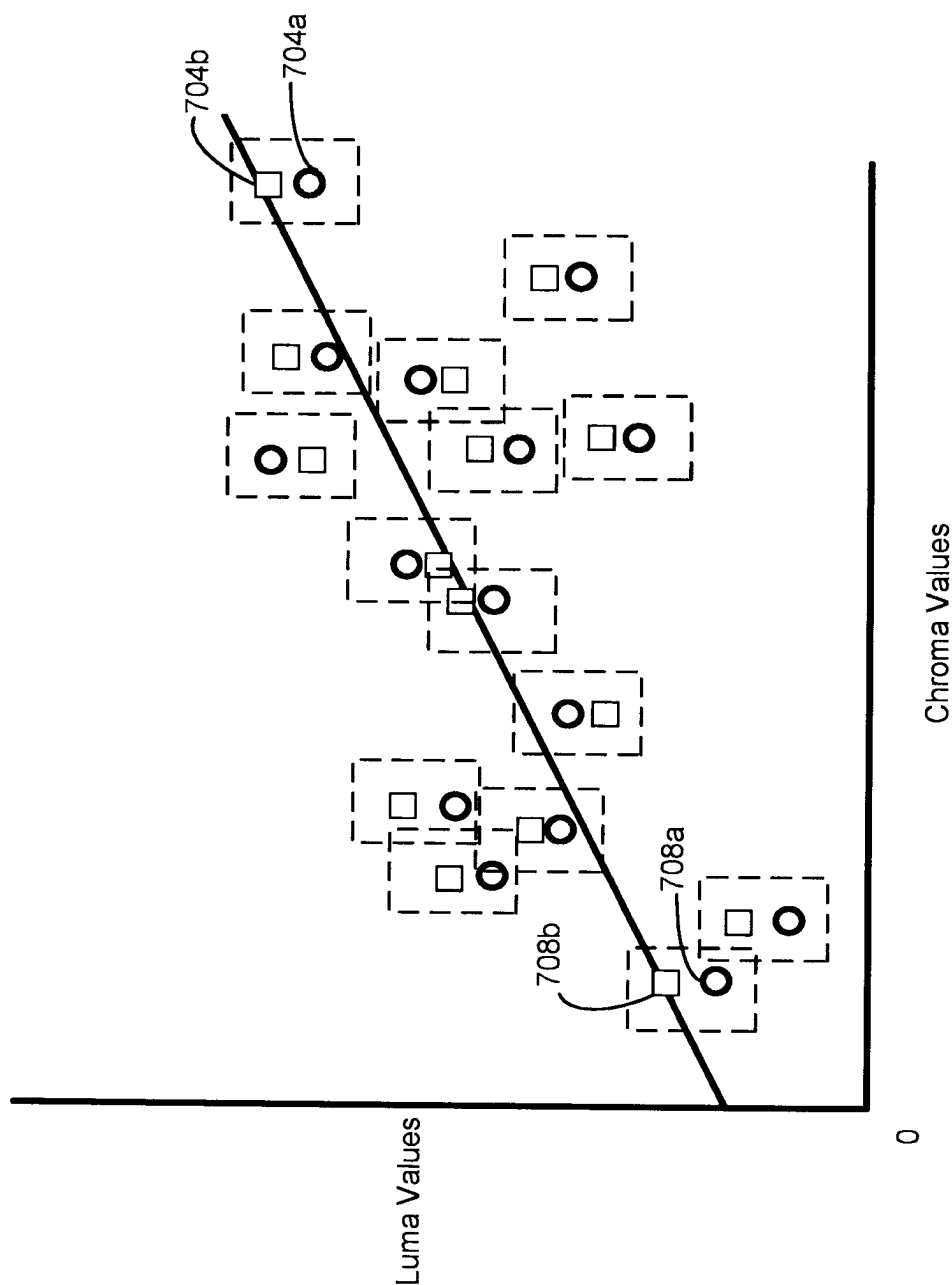


FIG. 7D

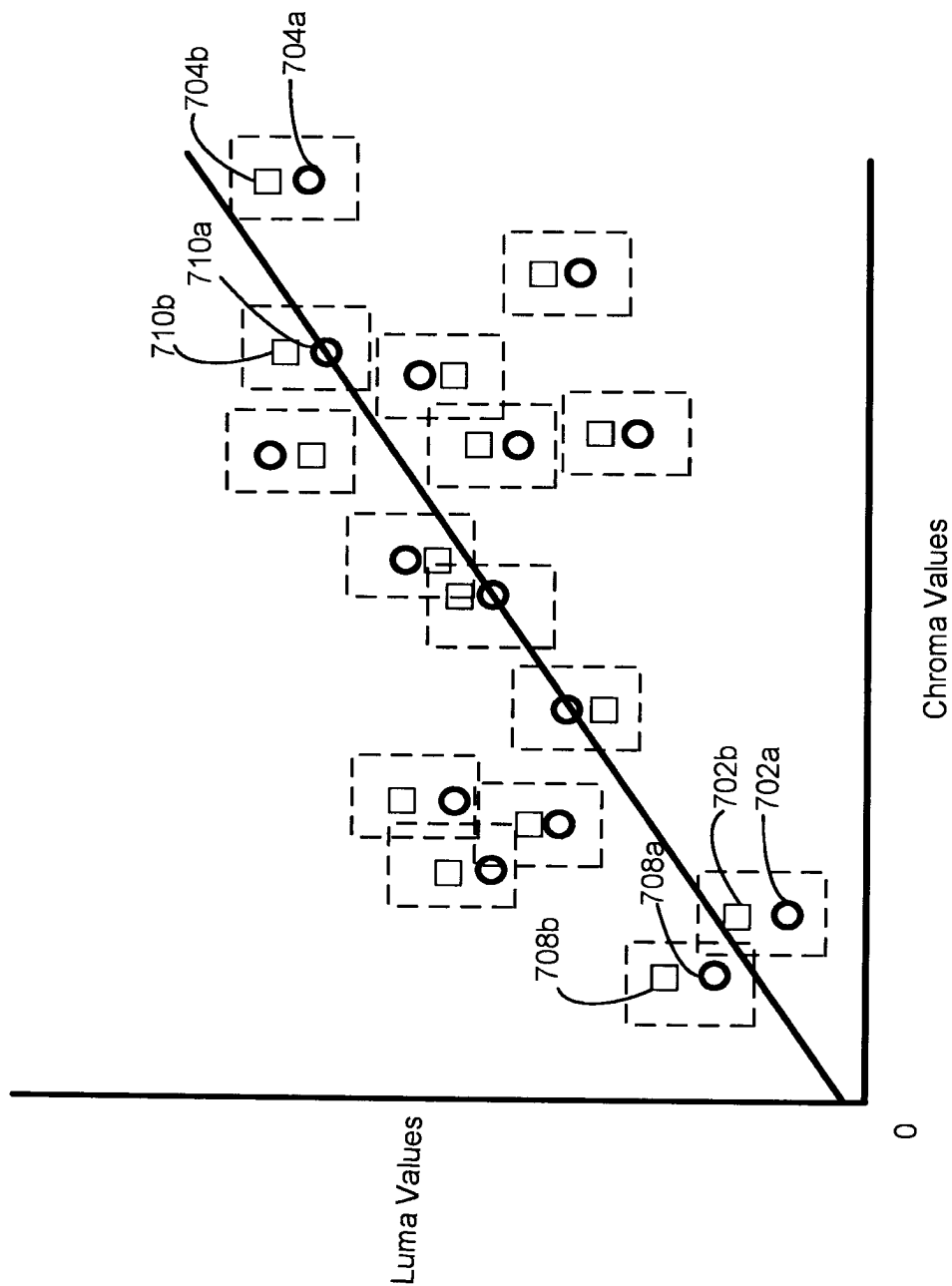


FIG. 7E

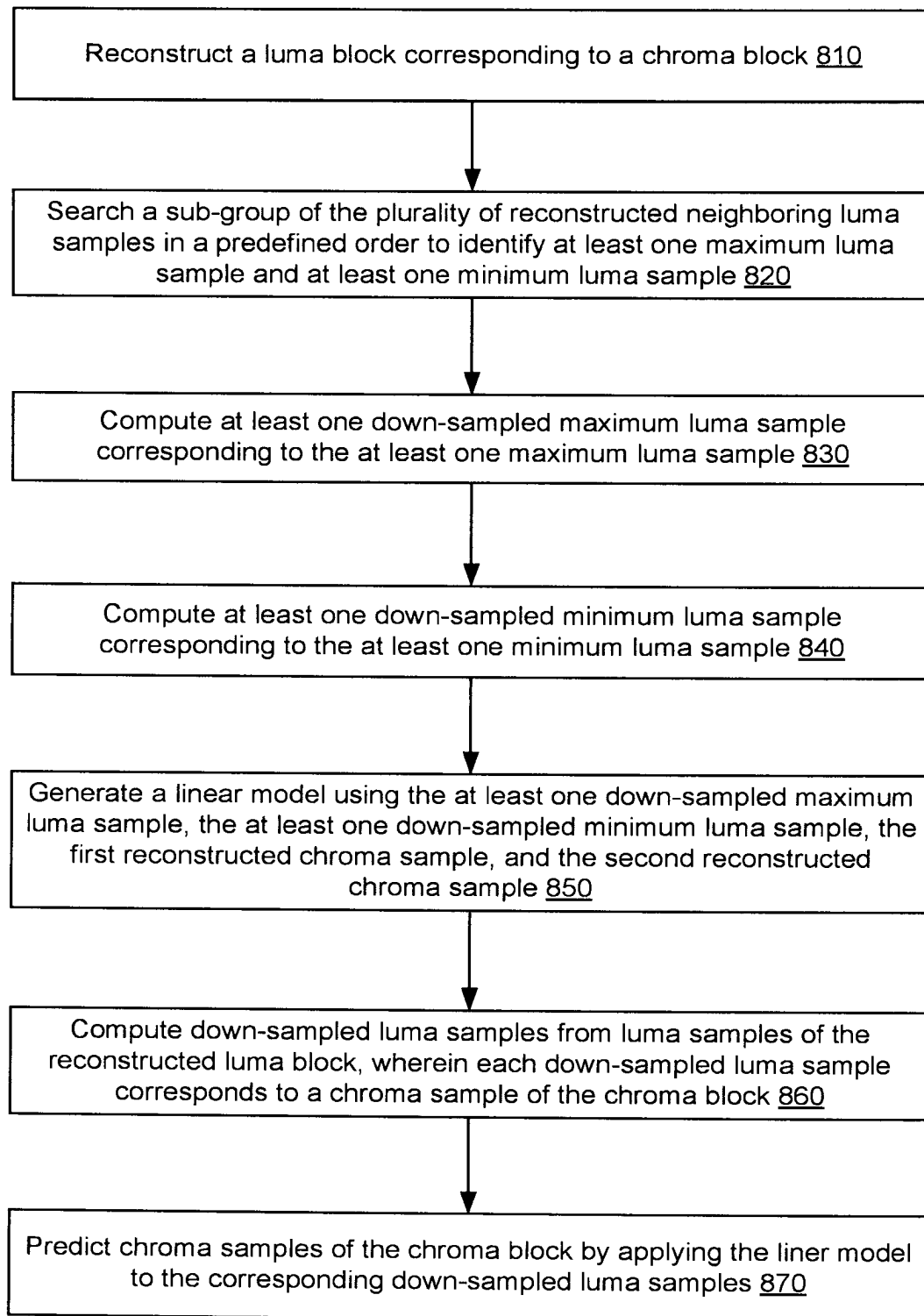


FIG. 8

SIMPLIFICATIONS OF CROSS-COMPONENT LINEAR MODEL

CROSS REFERENCE TO RELATED APPLICATIONS

This application is a continuation application of U.S. patent application Ser. No. 18/212,640, filed on Jun. 21, 2023, which is a continuation application of U.S. patent application Ser. No. 18/126,179, filed on Mar. 24, 2023, which is a continuation application of U.S. patent application Ser. No. 17/700,238, filed on Mar. 21, 2022, which is a continuation application of U.S. patent application Ser. No. 17/225,955, filed on Apr. 8, 2021, which is a continuation application of PCT Patent Application No. PCT/US2019/055208, filed on Oct. 8, 2019, which claims the benefit of U.S. Provisional Patent Application No. 62/742,806, filed Oct. 8, 2018, all of which are incorporated herein by reference in their entirety.

TECHNICAL FIELD

The present application generally relates to video data encoding and decoding, and in particular, to method and system of reconstructing a chroma block using a cross-component linear model during video data encoding and decoding.

BACKGROUND

Digital video is supported by a variety of electronic devices, such as digital televisions, laptop or desktop computers, tablet computers, digital cameras, digital recording devices, digital media players, video gaming consoles, smart phones, video teleconferencing devices, video streaming devices, etc. The electronic devices transmit, receive, encode, decode, and/or store digital video data by implementing video compression/decompression standards as defined by MPEG-4, ITU-T H.263, ITU-T H.264/MPEG-4, Part 10, Advanced Video Coding (AVC), High Efficiency Video Coding (HEVC), and Versatile Video Coding (VVC) standard. Video compression typically includes performing spatial (intra frame) prediction and/or temporal (inter frame) prediction to reduce or remove redundancy inherent in the video data. For block-based video coding, a video frame is partitioned into one or more slices, each slice having multiple video blocks, which may also be referred to as coding tree units (CTUs). Each CTU may contain one coding unit (CU) or recursively split into smaller CUs until the predefined minimum CU size is reached. Each CU (also named leaf CU) contains one or multiple transform units (TUs) and each CU also contains one or multiple prediction units (PUs). Each CU can be coded in either intra, inter or IBC modes. Video blocks in an intra coded (I) slice of a video frame are encoded using spatial prediction with respect to reference samples in neighbor blocks within the same video frame. Video blocks in an inter coded (P or B) slice of a video frame may use spatial prediction with respect to reference samples in neighbor blocks within the same video frame or temporal prediction with respect to reference samples in other previous and/or future reference video frames.

Spatial or temporal prediction based on a reference block that has been previously encoded, e.g., a neighbor block, results in a predictive block for a current video block to be coded. The process of finding the reference block may be accomplished by block matching algorithm. Residual data

representing pixel differences between the current block to be coded and the predictive block is referred to as a residual block or prediction errors. An inter-coded block is encoded according to a motion vector that points to a reference block in a reference frame forming the predictive block, and the residual block. The process of determining the motion vector is typically referred to as motion estimation. An intra coded block is encoded according to an intra prediction mode and the residual block. For further compression, the residual block is transformed from the pixel domain to a transform domain, e.g., frequency domain, resulting in residual transform coefficients, which may then be quantized. The quantized transform coefficients, initially arranged in a two-dimensional array, may be scanned to produce a one-dimensional vector of transform coefficients, and then entropy encoded into a video bitstream to achieve even more compression.

The encoded video bitstream is then saved in a computer-readable storage medium (e.g., flash memory) to be accessed by another electronic device with digital video capability or directly transmitted to the electronic device wired or wirelessly. The electronic device then performs video decompression (which is an opposite process to the video compression described above) by, e.g., parsing the encoded video bitstream to obtain syntax elements from the bitstream and reconstructing the digital video data to its original format from the encoded video bitstream based at least in part on the syntax elements obtained from the bitstream, and renders the reconstructed digital video data on a display of the electronic device.

With digital video quality going from high definition, to 4K×2K or even 8K×4K, the amount of video data to be encoded/decoded grows exponentially. It is a constant challenge in terms of how the video data can be encoded/decoded more efficiently while maintaining the image quality of the decoded video data.

SUMMARY

The present application describes implementations related to video data encoding and decoding and, more particularly, to system and method of reconstructing a chroma block using a cross-component linear model during video data encoding and decoding.

According to a first aspect of the present application, a method of decoding video data is performed at a computing device having one or more processors and memory storing a plurality of programs to be executed by the one or more processors. A computing device performs a method of reconstructing a luma block corresponding to a chroma block; searching a sub-group of a plurality of reconstructed neighboring luma samples in a predefined order to identify a maximum luma sample and a minimum luma sample; computing a down-sampled maximum luma sample corresponding to the maximum luma sample; computing a down-sampled minimum luma sample corresponding to the minimum luma sample; generating a linear model using the down-sampled maximum luma sample, the down-sampled minimum luma sample, the first reconstructed chroma sample, and the second reconstructed chroma sample; computing down-sampled luma samples from luma samples of the reconstructed luma block, wherein each down-sampled luma sample corresponds to a chroma sample of the chroma block; and predicting chroma samples of the chroma block by applying the linear model to the corresponding down-sampled luma samples.

According to a second aspect of the present application, a computing device includes one or more processors, memory and a plurality of programs stored in the memory. The programs, when executed by the one or more processors, cause the computing device to perform operations as described above.

According to a third aspect of the present application, a non-transitory computer readable storage medium stores a plurality of programs for execution by a computing device having one or more processors. The programs, when executed by the one or more processors, cause the computing device to perform operations as described above.

BRIEF DESCRIPTION OF DRAWINGS

The accompanying drawings, which are included to provide a further understanding of the implementations and are incorporated herein and constitute a part of the specification, illustrate the described implementations and together with the description serve to explain the underlying principles. Like reference numerals refer to corresponding parts.

FIG. 1 is a block diagram illustrating an exemplary video encoding and decoding system in accordance with some implementations of the present disclosure.

FIG. 2 is a block diagram illustrating an exemplary video encoder in accordance with some implementations of the present disclosure.

FIG. 3 is a block diagram illustrating an exemplary video decoder in accordance with some implementations of the present disclosure.

FIGS. 4A-4D are block diagrams illustrating how a frame is recursively quad-tree partitioned into multiple video blocks of different sizes in accordance with some implementations of the present disclosure.

FIG. 5A is a block diagram illustrating spatially neighboring and temporally collocated block positions of a current CU to be encoded in accordance with some implementations of the present disclosure.

FIG. 5B is a block diagram illustrating multi-threaded encoding of multiple rows of CTUs of a picture using wavefront parallel processing in accordance with some implementations of the present disclosure.

FIGS. 6A and 6B are block diagrams illustrating an exemplary reconstructed luma block and an exemplary associated chroma block, respectively, in accordance with some implementations of the present disclosure.

FIGS. 7A-7E illustrate various ways of using the cross-component linear model to derive a linear model between luma values and chroma values in accordance with some implementations of the present disclosure.

FIG. 8 is a flowchart illustrating an exemplary process by which a video codec implements the techniques of using a cross-component linear model to reconstruct chroma samples for a chroma block based on reconstructed luma samples from a luma block in accordance with some implementations of the present disclosure.

DETAILED DESCRIPTION

Reference will now be made in detail to specific implementations, examples of which are illustrated in the accompanying drawings. In the following detailed description, numerous non-limiting specific details are set forth in order to assist in understanding the subject matter presented herein. But it will be apparent to one of ordinary skill in the art that various alternatives may be used without departing from the scope of claims and the subject matter may be

practiced without these specific details. For example, it will be apparent to one of ordinary skill in the art that the subject matter presented herein can be implemented on many types of electronic devices with digital video capabilities.

FIG. 1 is a block diagram illustrating an exemplary system 10 for encoding and decoding video blocks in parallel in accordance with some implementations of the present disclosure. As shown in FIG. 1, system 10 includes a source device 12 that generates and encodes video data to be decoded at a later time by a destination device 14. Source device 12 and destination device 14 may comprise any of a wide variety of electronic devices, including desktop or laptop computers, tablet computers, smart phones, set-top boxes, digital televisions, cameras, display devices, digital media players, video gaming consoles, video streaming device, or the like. In some implementations, source device 12 and destination device 14 are equipped with wireless communication capabilities.

In some implementations, destination device 14 may receive the encoded video data to be decoded via a link 16. Link 16 may comprise any type of communication medium or device capable of moving the encoded video data from source device 12 to destination device 14. In one example, link 16 may comprise a communication medium to enable source device 12 to transmit the encoded video data directly to destination device 14 in real-time. The encoded video data may be modulated according to a communication standard, such as a wireless communication protocol, and transmitted to destination device 14. The communication medium may comprise any wireless or wired communication medium, such as a radio frequency (RF) spectrum or one or more physical transmission lines. The communication medium may form part of a packet-based network, such as a local area network, a wide-area network, or a global network such as the Internet. The communication medium may include routers, switches, base stations, or any other equipment that may be useful to facilitate communication from source device 12 to destination device 14.

In some other implementations, the encoded video data may be transmitted from output interface 22 to a storage device 32. Subsequently, the encoded video data in storage device 32 may be accessed by destination device 14 via input interface 28. Storage device 32 may include any of a variety of distributed or locally accessed data storage media such as a hard drive, Blu-ray discs, DVDs, CD-ROMs, flash memory, volatile or non-volatile memory, or any other suitable digital storage media for storing encoded video data. In a further example, storage device 32 may correspond to a file server or another intermediate storage device that may hold the encoded video data generated by source device 12. Destination device 14 may access the stored video data from storage device 32 via streaming or downloading. The file server may be any type of computer capable of storing encoded video data and transmitting the encoded video data to destination device 14. Exemplary file servers include a web server (e.g., for a website), an FTP server, network attached storage (NAS) devices, or a local disk drive. Destination device 14 may access the encoded video data through any standard data connection, including a wireless channel (e.g., a Wi-Fi connection), a wired connection (e.g., DSL, cable modem, etc.), or a combination of both that is suitable for accessing encoded video data stored on a file server. The transmission of encoded video data from storage device 32 may be a streaming transmission, a download transmission, or a combination of both.

As shown in FIG. 1, source device 12 includes a video source 18, a video encoder 20 and an output interface 22.

5

Video source **18** may include a source such as a video capture device, e.g., a video camera, a video archive containing previously captured video, a video feed interface to receive video from a video content provider, and/or a computer graphics system for generating computer graphics data as the source video, or a combination of such sources. As one example, if video source **18** is a video camera of a security surveillance system, source device **12** and destination device **14** may form camera phones or video phones. However, the implementations described in the present application may be applicable to video coding in general, and may be applied to wireless and/or wired applications.

The captured, pre-captured, or computer-generated video may be encoded by video encoder **20**. The encoded video data may be transmitted directly to destination device **14** via output interface **22** of source device **12**. The encoded video data may also (or alternatively) be stored onto storage device **32** for later access by destination device **14** or other devices, for decoding and/or playback. Output interface **22** may further include a modem and/or a transmitter.

Destination device **14** includes an input interface **28**, a video decoder **30**, and a display device **34**. Input interface **28** may include a receiver and/or a modem and receive the encoded video data over link **16**. The encoded video data communicated over link **16**, or provided on storage device **32**, may include a variety of syntax elements generated by video encoder **20** for use by video decoder **30** in decoding the video data. Such syntax elements may be included within the encoded video data transmitted on a communication medium, stored on a storage medium, or stored a file server.

In some implementations, destination device **14** may include a display device **34**, which can be an integrated display device and an external display device that is configured to communicate with destination device **14**. Display device **34** displays the decoded video data to a user, and may comprise any of a variety of display devices such as a liquid crystal display (LCD), a plasma display, an organic light emitting diode (OLED) display, or another type of display device.

Video encoder **20** and video decoder **30** may operate according to proprietary or industry standards, such as VVC, HEVC, MPEG-4, Part 10, Advanced Video Coding (AVC), or extensions of such standards. It should be understood that the present application is not limited to a specific video coding/decoding standard and may be applicable to other video coding/decoding standards. It is generally contemplated that video encoder **20** of source device **12** may be configured to encode video data according to any of these current or future standards. Similarly, it is also generally contemplated that video decoder **30** of destination device **14** may be configured to decode video data according to any of these current or future standards.

Video encoder **20** and video decoder **30** each may be implemented as any of a variety of suitable encoder circuitry, such as one or more microprocessors, digital signal processors (DSPs), application specific integrated circuits (ASICs), field programmable gate arrays (FPGAs), discrete logic, software, hardware, firmware or any combinations thereof. When implemented partially in software, an electronic device may store instructions for the software in a suitable, non-transitory computer-readable medium and execute the instructions in hardware using one or more processors to perform the video coding/decoding operations disclosed in the present disclosure. Each of video encoder **20** and video decoder **30** may be included in one or more

6

encoders or decoders, either of which may be integrated as part of a combined encoder/decoder (CODEC) in a respective device.

FIG. 2 is a block diagram illustrating an exemplary video encoder **20** in accordance with some implementations described in the present application. Video encoder **20** may perform intra and inter predictive coding of video blocks within video frames. Intra predictive coding relies on spatial prediction to reduce or remove spatial redundancy in video data within a given video frame or picture. Inter predictive coding relies on temporal prediction to reduce or remove temporal redundancy in video data within adjacent video frames or pictures of a video sequence.

As shown in FIG. 2, video encoder **20** includes video data memory **40**, prediction processing unit **41**, decoded picture buffer (DPB) **64**, summer **50**, transform processing unit **52**, quantization unit **54**, and entropy encoding unit **56**. Prediction processing unit **41** further includes motion estimation unit **42**, motion compensation unit **44**, partition unit **45**, intra prediction processing unit **46**, and intra block copy (BC) unit **48**. In some implementations, video encoder **20** also includes inverse quantization unit **58**, inverse transform processing unit **60**, and summer **62** for video block reconstruction. A deblocking filter (not shown) may be positioned between summer **62** and DPB **64** to filter block boundaries to remove blockiness artifacts from reconstructed video. An in loop filter (not shown) may also be used in addition to the deblocking filter to filter the output of summer **62**. Video encoder **20** may take the form of a fixed or programmable hardware unit or may be divided among one or more of the illustrated fixed or programmable hardware units.

Video data memory **40** may store video data to be encoded by the components of video encoder **20**. The video data in video data memory **40** may be obtained, for example, from video source **18**. DPB **64** is a buffer that stores reference video data for use in encoding video data by video encoder **20** (e.g., in intra or inter predictive coding modes). Video data memory **40** and DPB **64** may be formed by any of a variety of memory devices. In various examples, video data memory **40** may be on-chip with other components of video encoder **20**, or off-chip relative to those components.

As shown in FIG. 2, after receiving video data, partition unit **45** within prediction processing unit **41** partitions the video data into video blocks. This partitioning may also include partitioning a video frame into slices, tiles, or other larger coding units (CUs) according to a predefined splitting structures such as quad-tree structure associated with the video data. The video frame may be divided into multiple video blocks (or sets of video blocks referred to as tiles). Prediction processing unit **41** may select one of a plurality of possible predictive coding modes, such as one of a plurality of intra predictive coding modes or one of a plurality of inter predictive coding modes, for the current video block based on error results (e.g., coding rate and the level of distortion). Prediction processing unit **41** may provide the resulting intra or inter prediction coded block to summer **50** to generate a residual block and to summer **62** to reconstruct the encoded block for use as part of a reference frame subsequently. Prediction processing unit **41** also provides syntax elements, such as motion vectors, intra-mode indicators, partition information, and other such syntax information, to entropy encoding unit **56**.

In order to select an appropriate intra predictive coding mode for the current video block, intra prediction processing unit **46** within prediction processing unit **41** may perform intra predictive coding of the current video block relative to one or more neighbor blocks in the same frame as the current

block to be coded to provide spatial prediction. Motion estimation unit **42** and motion compensation unit **44** within prediction processing unit **41** perform inter predictive coding of the current video block relative to one or more predictive blocks in one or more reference frames to provide temporal prediction. Video encoder **20** may perform multiple coding passes, e.g., to select an appropriate coding mode for each block of video data.

In some implementations, motion estimation unit **42** determines the inter prediction mode for a current video frame by generating a motion vector, which indicates the displacement of a prediction unit (PU) of a video block within the current video frame relative to a predictive block within a reference video frame, according to a predetermined pattern within a sequence of video frames. Motion estimation, performed by motion estimation unit **42**, is the process of generating motion vectors, which estimate motion for video blocks. A motion vector, for example, may indicate the displacement of a PU of a video block within a current video frame or picture relative to a predictive block within a reference frame (or other coded unit) relative to the current block being coded within the current frame (or other coded unit). The predetermined pattern may designate video frames in the sequence as P frames or B frames. Intra BC unit **48** may determine vectors, e.g., block vectors, for intra BC coding in a manner similar to the determination of motion vectors by motion estimation unit **42** for inter prediction, or may utilize motion estimation unit **42** to determine the block vector.

A predictive block is a block of a reference frame that is deemed as closely matching the PU of the video block to be coded in terms of pixel difference, which may be determined by sum of absolute difference (SAD), sum of square difference (SSD), or other difference metrics. In some implementations, video encoder **20** may calculate values for sub-integer pixel positions of reference frames stored in DPB **64**. For example, video encoder **20** may interpolate values of one-quarter pixel positions, one-eighth pixel positions, or other fractional pixel positions of the reference frame. Therefore, motion estimation unit **42** may perform a motion search relative to the full pixel positions and fractional pixel positions and output a motion vector with fractional pixel precision.

Motion estimation unit **42** calculates a motion vector for a PU of a video block in an inter prediction coded frame by comparing the position of the PU to the position of a predictive block of a reference frame selected from a first reference frame list (List 0) or a second reference frame list (List 1), each of which identifies one or more reference frames stored in DPB **64**. Motion estimation unit **42** sends the calculated motion vector to motion compensation unit **44** and then to entropy encoding unit **56**.

Motion compensation, performed by motion compensation unit **44**, may involve fetching or generating the predictive block based on the motion vector determined by motion estimation unit **42**. Upon receiving the motion vector for the PU of the current video block, motion compensation unit **44** may locate a predictive block to which the motion vector points in one of the reference frame lists, retrieve the predictive block from DPB **64**, and forward the predictive block to summer **50**. Summer **50** then forms a residual video block of pixel difference values by subtracting pixel values of the predictive block provided by motion compensation unit **44** from the pixel values of the current video block being coded. The pixel difference values forming the residual video block may include luma or chroma difference components or both. Motion compensation unit **44** may also

generate syntax elements associated with the video blocks of a video frame for use by video decoder **30** in decoding the video blocks of the video frame. The syntax elements may include, for example, syntax elements defining the motion vector used to identify the predictive block, any flags indicating the prediction mode, or any other syntax information described herein. Note that motion estimation unit **42** and motion compensation unit **44** may be highly integrated, but are illustrated separately for conceptual purposes.

In some implementations, intra BC unit **48** may generate vectors and fetch predictive blocks in a manner similar to that described above in connection with motion estimation unit **42** and motion compensation unit **44**, but with the predictive blocks being in the same frame as the current block being coded and with the vectors being referred to as block vectors as opposed to motion vectors. In particular, intra BC unit **48** may determine an intra-prediction mode to use to encode a current block. In some examples, intra BC unit **48** may encode a current block using various intra-prediction modes, e.g., during separate encoding passes, and test their performance through rate-distortion analysis. Next, intra BC unit **48** may select, among the various tested intra-prediction modes, an appropriate intra-prediction mode to use and generate an intra-mode indicator accordingly. For example, intra BC unit **48** may calculate rate-distortion values using a rate-distortion analysis for the various tested intra-prediction modes, and select the intra-prediction mode having the best rate-distortion characteristics among the tested modes as the appropriate intra-prediction mode to use. Rate-distortion analysis generally determines an amount of distortion (or error) between an encoded block and an original, unencoded block that was encoded to produce the encoded block, as well as a bitrate (i.e., a number of bits) used to produce the encoded block. Intra BC unit **48** may calculate ratios from the distortions and rates for the various encoded blocks to determine which intra-prediction mode exhibits the best rate-distortion value for the block.

In other examples, intra BC unit **48** may use motion estimation unit **42** and motion compensation unit **44**, in whole or in part, to perform such functions for Intra BC prediction according to the implementations described herein. In either case, for Intra block copy, a predictive block may be a block that is deemed as closely matching the block to be coded, in terms of pixel difference, which may be determined by sum of absolute difference (SAD), sum of squared difference (SSD), or other difference metrics, and identification of the predictive block may include calculation of values for sub-integer pixel positions.

Whether the predictive block is from the same frame according to intra prediction, or a different frame according to inter prediction, video encoder **20** may form a residual video block by subtracting pixel values of the predictive block from the pixel values of the current video block being coded, forming pixel difference values. The pixel difference values forming the residual video block may include both luma and chroma component differences.

Intra prediction processing unit **46** may intra-predict a current video block, as an alternative to the inter-prediction performed by motion estimation unit **42** and motion compensation unit **44**, or the intra block copy prediction performed by intra BC unit **48**, as described above. In particular, intra prediction processing unit **46** may determine an intra prediction mode to use to encode a current block. To do so, intra prediction processing unit **46** may encode a current block using various intra prediction modes, e.g., during separate encoding passes, and intra prediction processing

unit **46** (or a mode select unit, in some examples) may select an appropriate intra prediction mode to use from the tested intra prediction modes. Intra prediction processing unit **46** may provide information indicative of the selected intra-prediction mode for the block to entropy encoding unit **56**. Entropy encoding unit **56** may encode the information indicating the selected intra-prediction mode in the bitstream.

After prediction processing unit **41** determines the predictive block for the current video block via either inter prediction or intra prediction, summer **50** forms a residual video block by subtracting the predictive block from the current video block. The residual video data in the residual block may be included in one or more transform units (TUs) and is provided to transform processing unit **52**. Transform processing unit **52** transforms the residual video data into residual transform coefficients using a transform, such as a discrete cosine transform (DCT) or a conceptually similar transform.

Transform processing unit **52** may send the resulting transform coefficients to quantization unit **54**. Quantization unit **54** quantizes the transform coefficients to further reduce bit rate. The quantization process may also reduce the bit depth associated with some or all of the coefficients. The degree of quantization may be modified by adjusting a quantization parameter. In some examples, quantization unit **54** may then perform a scan of a matrix including the quantized transform coefficients. Alternatively, entropy encoding unit **56** may perform the scan.

Following quantization, entropy encoding unit **56** entropy encodes the quantized transform coefficients into a video bitstream using, e.g., context adaptive variable length coding (CAVLC), context adaptive binary arithmetic coding (CABAC), syntax-based context-adaptive binary arithmetic coding (SBAC), probability interval partitioning entropy (PIPE) coding or another entropy encoding methodology or technique. The encoded bitstream may then be transmitted to video decoder **30**, or archived in storage device **32** for later transmission to or retrieval by video decoder **30**. Entropy encoding unit **56** may also entropy encode the motion vectors and the other syntax elements for the current video frame being coded.

Inverse quantization unit **58** and inverse transform processing unit **60** apply inverse quantization and inverse transformation, respectively, to reconstruct the residual video block in the pixel domain for generating a reference block for prediction of other video blocks. As noted above, motion compensation unit **44** may generate a motion compensated predictive block from one or more reference blocks of the frames stored in DPB **64**. Motion compensation unit **44** may also apply one or more interpolation filters to the predictive block to calculate sub-integer pixel values for use in motion estimation.

Summer **62** adds the reconstructed residual block to the motion compensated predictive block produced by motion compensation unit **44** to produce a reference block for storage in DPB **64**. The reference block may then be used by intra BC unit **48**, motion estimation unit **42** and motion compensation unit **44** as a predictive block to inter predict another video block in a subsequent video frame.

FIG. 3 is a block diagram illustrating an exemplary video decoder **30** in accordance with some implementations of the present application. Video decoder **30** includes video data memory **79**, entropy decoding unit **80**, prediction processing unit **81**, inverse quantization unit **86**, inverse transform processing unit **88**, summer **90**, and DPB **92**. Prediction processing unit **81** further includes motion compensation

unit **82**, intra prediction unit **84**, and intra BC unit **85**. Video decoder **30** may perform a decoding process generally reciprocal to the encoding process described above with respect to video encoder **20** in connection with FIG. 2. For example, motion compensation unit **82** may generate prediction data based on motion vectors received from entropy decoding unit **80**, while intra-prediction unit **84** may generate prediction data based on intra-prediction mode indicators received from entropy decoding unit **80**.

In some examples, a unit of video decoder **30** may be tasked to perform the implementations of the present application. Also, in some examples, the implementations of the present disclosure may be divided among one or more of the units of video decoder **30**. For example, intra BC unit **85** may perform the implementations of the present application, alone, or in combination with other units of video decoder **30**, such as motion compensation unit **82**, intra prediction unit **84**, and entropy decoding unit **80**. In some examples, video decoder **30** may not include intra BC unit **85** and the functionality of intra BC unit **85** may be performed by other components of prediction processing unit **81**, such as motion compensation unit **82**.

Video data memory **79** may store video data, such as an encoded video bitstream, to be decoded by the other components of video decoder **30**. The video data stored in video data memory **79** may be obtained, for example, from storage device **32**, from a local video source, such as a camera, via wired or wireless network communication of video data, or by accessing physical data storage media (e.g., a flash drive or hard disk). Video data memory **79** may include a coded picture buffer (CPB) that stores encoded video data from an encoded video bitstream. Decoded picture buffer (DPB) **92** of video decoder **30** stores reference video data for use in decoding video data by video decoder **30** (e.g., in intra or inter predictive coding modes). Video data memory **79** and DPB **92** may be formed by any of a variety of memory devices, such as dynamic random access memory (DRAM), including synchronous DRAM (SDRAM), magneto-resistive RAM (MRAM), resistive RAM (RRAM), or other types of memory devices. For illustrative purpose, video data memory **79** and DPB **92** are depicted as two distinct components of video decoder **30** in FIG. 3. But it will be apparent to one skilled in the art that video data memory **79** and DPB **92** may be provided by the same memory device or separate memory devices. In some examples, video data memory **79** may be on-chip with other components of video decoder **30**, or off-chip relative to those components.

During the decoding process, video decoder **30** receives an encoded video bitstream that represents video blocks of an encoded video frame and associated syntax elements. Video decoder **30** may receive the syntax elements at the video frame level and/or the video block level. Entropy decoding unit **80** of video decoder **30** entropy decodes the bitstream to generate quantized coefficients, motion vectors or intra-prediction mode indicators, and other syntax elements. Entropy decoding unit **80** then forwards the motion vectors and other syntax elements to prediction processing unit **81**.

When the video frame is coded as an intra predictive coded (I) frame or for intra coded predictive blocks in other types of frames, intra prediction unit **84** of prediction processing unit **81** may generate prediction data for a video block of the current video frame based on a signaled intra prediction mode and reference data from previously decoded blocks of the current frame.

When the video frame is coded as an inter-predictive coded (i.e., B or P) frame, motion compensation unit **82** of

11

prediction processing unit **81** produces one or more predictive blocks for a video block of the current video frame based on the motion vectors and other syntax elements received from entropy decoding unit **80**. Each of the predictive blocks may be produced from a reference frame within one of the reference frame lists. Video decoder **30** may construct the reference frame lists, List 0 and List 1, using default construction techniques based on reference frames stored in DPB **92**.

In some examples, when the video block is coded according to the intra BC mode described herein, intra BC unit **85** of prediction processing unit **81** produces predictive blocks for the current video block based on block vectors and other syntax elements received from entropy decoding unit **80**. The predictive blocks may be within a reconstructed region of the same picture as the current video block defined by video encoder **20**.

Motion compensation unit **82** and/or intra BC unit **85** determines prediction information for a video block of the current video frame by parsing the motion vectors and other syntax elements, and then uses the prediction information to produce the predictive blocks for the current video block being decoded. For example, motion compensation unit **82** uses some of the received syntax elements to determine a prediction mode (e.g., intra or inter prediction) used to code video blocks of the video frame, an inter prediction frame type (e.g., B or P), construction information for one or more of the reference frame lists for the frame, motion vectors for each inter predictive encoded video block of the frame, inter prediction status for each inter predictive coded video block of the frame, and other information to decode the video blocks in the current video frame.

Similarly, intra BC unit **85** may use some of the received syntax elements, e.g., a flag, to determine that the current video block was predicted using the intra BC mode, construction information of which video blocks of the frame are within the reconstructed region and should be stored in DPB **92**, block vectors for each intra BC predicted video block of the frame, intra BC prediction status for each intra BC predicted video block of the frame, and other information to decode the video blocks in the current video frame.

Motion compensation unit **82** may also perform interpolation using the interpolation filters as used by video encoder **20** during encoding of the video blocks to calculate interpolated values for sub-integer pixels of reference blocks. In this case, motion compensation unit **82** may determine the interpolation filters used by video encoder **20** from the received syntax elements and use the interpolation filters to produce predictive blocks.

Inverse quantization unit **86** inverse quantizes the quantized transform coefficients provided in the bitstream and entropy decoded by entropy decoding unit **80** using the same quantization parameter calculated by video encoder **20** for each video block in the video frame to determine a degree of quantization. Inverse transform processing unit **88** applies an inverse transform, e.g., an inverse DCT, an inverse integer transform, or a conceptually similar inverse transform process, to the transform coefficients in order to reconstruct the residual blocks in the pixel domain.

After motion compensation unit **82** or intra BC unit **85** generates the predictive block for the current video block based on the vectors and other syntax elements, summer **90** reconstructs decoded video block for the current video block by summing the residual block from inverse transform processing unit **88** and a corresponding predictive block generated by motion compensation unit **82** and intra BC unit **85**. An in-loop filter (not pictured) may be positioned

12

between summer **90** and DPB **92** to further process the decoded video block. The decoded video blocks in a given frame are then stored in DPB **92**, which stores reference frames used for subsequent motion compensation of next video blocks. DPB **92**, or a memory device separate from DPB **92**, may also store decoded video for later presentation on a display device, such as display device **34** of FIG. 1.

In a typical video coding process, a video sequence typically includes an ordered set of frames or pictures. Each frame may include three sample arrays, denoted SL, SCb, and SCr. SL is a two-dimensional array of luma samples. SCb is a two-dimensional array of Cb chroma samples. SCr is a two-dimensional array of Cr chroma samples. In other instances, a frame may be monochrome and therefore includes only one two-dimensional array of luma samples.

As shown in FIG. 4A, video encoder **20** (or more specifically partition unit **45**) generates an encoded representation of a frame by first partitioning the frame into a set of coding tree units (CTUs). A video frame may include an integer number of CTUs ordered consecutively in a raster scan order from left to right and from top to bottom. Each CTU is a largest logical coding unit and the width and height of the CTU are signaled by the video encoder **20** in a sequence parameter set, such that all the CTUs in a video sequence have the same size being one of 128×128, 64×64, 32×32, and 16×16. But it should be noted that the present application is not necessarily limited to a particular size. As shown in FIG. 4B, each CTU may comprise one coding tree block (CTB) of luma samples, two corresponding coding tree blocks of chroma samples, and syntax elements used to code the samples of the coding tree blocks. The syntax elements describe properties of different types of units of a coded block of pixels and how the video sequence can be reconstructed at the video decoder **30**, including inter or intra prediction, intra prediction mode, motion vectors, and other parameters. In monochrome pictures or pictures having three separate color planes, a CTU may comprise a single coding tree block and syntax elements used to code the samples of the coding tree block. A coding tree block may be an N×N block of samples.

To achieve a better performance, video encoder **20** may recursively perform tree partitioning such as binary-tree partitioning, quad-tree partitioning or a combination of both on the coding tree blocks of the CTU and divide the CTU into smaller coding units (CUs). As depicted in FIG. 4C, the 64×64 CTU **400** is first divided into four smaller CU, each having a block size of 32×32. Among the four smaller CUs, CU **410** and CU **420** are each divided into four CUs of 16×16 by block size. The two 16×16 CUs **430** and **440** are each further divided into four CUs of 8×8 by block size. FIG. 4D depicts a quad-tree data structure illustrating the end result of the partition process of the CTU **400** as depicted in FIG. 4C, each leaf node of the quad-tree corresponding to one CU of a respective size ranging from 32×32 to 8×8. Like the CTU depicted in FIG. 4B, each CU may comprise a coding block (CB) of luma samples and two corresponding coding blocks of chroma samples of a frame of the same size, and syntax elements used to code the samples of the coding blocks. In monochrome pictures or pictures having three separate color planes, a CU may comprise a single coding block and syntax structures used to code the samples of the coding block.

In some implementations, video encoder **20** may further partition a coding block of a CU into one or more M×N prediction blocks (PB). A prediction block is a rectangular (square or non-square) block of samples on which the same prediction, inter or intra, is applied. A prediction unit (PU)

13

of a CU may comprise a prediction block of luma samples, two corresponding prediction blocks of chroma samples, and syntax elements used to predict the prediction blocks. In monochrome pictures or pictures having three separate color planes, a PU may comprise a single prediction block and syntax structures used to predict the prediction block. Video encoder 20 may generate predictive luma, Cb, and Cr blocks for luma, Cb, and Cr prediction blocks of each PU of the CU.

Video encoder 20 may use intra prediction or inter prediction to generate the predictive blocks for a PU. If video encoder 20 uses intra prediction to generate the predictive blocks of a PU, video encoder 20 may generate the predictive blocks of the PU based on decoded samples of the frame associated with the PU. If video encoder 20 uses inter prediction to generate the predictive blocks of a PU, video encoder 20 may generate the predictive blocks of the PU based on decoded samples of one or more frames other than the frame associated with the PU.

After video encoder 20 generates predictive luma, Cb, and Cr blocks for one or more PUs of a CU, video encoder 20 may generate a luma residual block for the CU by subtracting the CU's predictive luma blocks from its original luma coding block such that each sample in the CU's luma residual block indicates a difference between a luma sample in one of the CU's predictive luma blocks and a corresponding sample in the CU's original luma coding block. Similarly, video encoder 20 may generate a Cb residual block and a Cr residual block for the CU, respectively, such that each sample in the CU's Cb residual block indicates a difference between a Cb sample in one of the CU's predictive Cb blocks and a corresponding sample in the CU's original Cb coding block and each sample in the CU's Cr residual block may indicate a difference between a Cr sample in one of the CU's predictive Cr blocks and a corresponding sample in the CU's original Cr coding block.

Furthermore, as illustrated in FIG. 4C, video encoder 20 may use quad-tree partitioning to decompose the luma, Cb, and Cr residual blocks of a CU into one or more luma, Cb, and Cr transform blocks. A transform block is a rectangular (square or non-square) block of samples on which the same transform is applied. A transform unit (TU) of a CU may comprise a transform block of luma samples, two corresponding transform blocks of chroma samples, and syntax elements used to transform the transform block samples. Thus, each TU of a CU may be associated with a luma transform block, a Cb transform block, and a Cr transform block. In some examples, the luma transform block associated with the TU may be a sub-block of the CU's luma residual block. The Cb transform block may be a sub-block of the CU's Cb residual block. The Cr transform block may be a sub-block of the CU's Cr residual block. In monochrome pictures or pictures having three separate color planes, a TU may comprise a single transform block and syntax structures used to transform the samples of the transform block.

Video encoder 20 may apply one or more transforms to a luma transform block of a TU to generate a luma coefficient block for the TU. A coefficient block may be a two-dimensional array of transform coefficients. A transform coefficient may be a scalar quantity. Video encoder 20 may apply one or more transforms to a Cb transform block of a TU to generate a Cb coefficient block for the TU. Video encoder 20 may apply one or more transforms to a Cr transform block of a TU to generate a Cr coefficient block for the TU.

After generating a coefficient block (e.g., a luma coefficient block, a Cb coefficient block or a Cr coefficient block),

14

video encoder 20 may quantize the coefficient block. Quantization generally refers to a process in which transform coefficients are quantized to possibly reduce the amount of data used to represent the transform coefficients, providing further compression. After video encoder 20 quantizes a coefficient block, video encoder 20 may entropy encode syntax elements indicating the quantized transform coefficients. For example, video encoder 20 may perform Context-Adaptive Binary Arithmetic Coding (CABAC) on the syntax elements indicating the quantized transform coefficients. Finally, video encoder 20 may output a bitstream that includes a sequence of bits that forms a representation of coded frames and associated data, which is either saved in storage device 32 or transmitted to destination device 14.

After receiving a bitstream generated by video encoder 20, video decoder 30 may parse the bitstream to obtain syntax elements from the bitstream. Video decoder 30 may reconstruct the frames of the video data based at least in part on the syntax elements obtained from the bitstream. The process of reconstructing the video data is generally reciprocal to the encoding process performed by video encoder 20. For example, video decoder 30 may perform inverse transforms on the coefficient blocks associated with TUs of a current CU to reconstruct residual blocks associated with the TUs of the current CU. Video decoder 30 also reconstructs the coding blocks of the current CU by adding the samples of the predictive blocks for PUs of the current CU to corresponding samples of the transform blocks of the TUs of the current CU. After reconstructing the coding blocks for each CU of a frame, video decoder 30 may reconstruct the frame.

As noted above, video coding achieves video compression using primarily two modes, i.e., intra-frame prediction (or intra-prediction) and inter-frame prediction (or inter-prediction). It is noted that IBC could be regarded as either intra-frame prediction or a third mode. Between the two modes, inter-frame prediction contributes more to the coding efficiency than intra-frame prediction because of the use of motion vectors for predicting a current video block from a reference video block.

But with the ever improving video data capturing technology and more refined video block size for preserving details in the video data, the amount of data required for representing motion vectors for a current frame also increases substantially. One way of overcoming this challenge is to benefit from the fact that not only a group of neighboring CUs in both the spatial and temporal domains have similar video data for predicting purpose but the motion vectors between these neighboring CUs are also similar. Therefore, it is possible to use the motion information of spatially neighboring CUs and/or temporally collocated CUs as an approximation of the motion information (e.g., motion vector) of a current CU by exploring their spatial and temporal correlation, which is also referred to as "motion vector predictor" (MVP) of the current CU.

Instead of encoding, into the video bitstream, an actual motion vector of the current CU determined by motion estimation unit 42 as described above in connection with FIG. 2, the motion vector predictor of the current CU is subtracted from the actual motion vector of the current CU to produce a motion vector difference (MVD) for the current CU. By doing so, there is no need to encode the motion vector determined by motion estimation unit 42 for each CU of a frame into the video bitstream and the amount of data used for representing motion information in the video bitstream can be significantly decreased.

15

Like the process of choosing a predictive block in a reference frame during inter-frame prediction of a code block, a set of rules need to be adopted by both video encoder 20 and video decoder 30 for constructing a motion vector candidate list (also known as a “merge list”) for a current CU using those potential candidate motion vectors associated with spatially neighboring CUs and/or temporally collocated CUs of the current CU and then selecting one member from the motion vector candidate list as a motion vector predictor for the current CU. By doing so, there is no need to transmit the motion vector candidate list itself between video encoder 20 and video decoder 30 and an index of the selected motion vector predictor within the motion vector candidate list is sufficient for video encoder 20 and video decoder 30 to use the same motion vector predictor within the motion vector candidate list for encoding and decoding the current CU.

In some implementations, each inter-prediction CU has three motion vector prediction modes including inter (which is also referred to as “advanced motion vector prediction” (AMVP)), skip, and merge for constructing the motion vector candidate list. Under each mode, one or more motion vector candidates may be added to the motion vector candidate list according to the algorithms described below. Ultimately one of them in the candidate list is used as the best motion vector predictor of the inter-prediction CU to be encoded into the video bitstream by video encoder 20 or decoded from the video bitstream by video decoder 30. To find the best motion vector predictor from the candidate list, a motion vector competition (MVC) scheme is introduced to select a motion vector from a given candidate set of motion vectors, i.e., the motion vector candidate list, that includes spatial and temporal motion vector candidates.

In addition to deriving motion vector predictor candidates from spatially neighboring or temporally collocated CUs, the motion vector predictor candidates can also be derived from the so-called “history-based motion vector prediction” (HMVP) table. The HMVP table houses a predefined number of motion vector predictors, each having been used for encoding/decoding a particular CU of the same row of CTUs (or sometimes the same CTU). Because of the spatial/temporal proximity of these CUs, there is a high likelihood that one of the motion vector predictors in the HMVP table may be reused for encoding/decoding different CUs within the same row of CTUs. Therefore, it is possible to achieve a higher code efficiency by including the HMVP table in the process of constructing the motion vector candidate list.

In some implementations, the HMVP table has a fixed length (e.g., 5) and is managed in a quasi-First-In-First-Out (FIFO) manner. For example, a motion vector is reconstructed for a CU when decoding one inter-coded block of the CU. The HMVP table is updated on-the-fly with the reconstructed motion vector because such motion vector could be the motion vector predictor of a subsequent CU. When updating the HMVP table, there are two scenarios: (i) the reconstructed motion vector is different from other existing motion vectors in the HMVP table or (ii) the reconstructed motion vector is the same as one of the existing motion vectors in the HMVP table. For the first scenario, the reconstructed motion vector is added to the HMVP table as the newest one if the HMVP table is not full. If the HMVP table is already full, the oldest motion vector in the HMVP table needs to be removed from the HMVP table first before the reconstructed motion vector is added as the newest one. In other words, the HMVP table in this case is similar to a FIFO buffer such that the motion information located at the head of the FIFO buffer and associated with

16

another previously inter-coded block is shifted out of the buffer so that the reconstructed motion vector is appended to the tail of the FIFO buffer as the newest member in the HMVP table. For the second scenario, the existing motion vector in the HMVP table that is substantially identical to the reconstructed motion vector is removed from the HMVP table before the reconstructed motion vector is added to the HMVP table as the newest one. If the HMVP table is also maintained in the form of a FIFO buffer, the motion vector predictors after the identical motion vector in the HMVP table are shifted forward by one element to occupy the space left by the removed motion vector and the reconstructed motion vector is then appended to the tail of the FIFO buffer as the newest member in the HMVP table.

The motion vectors in the HMVP table could be added to the motion vector candidate lists under different prediction modes such as AMVP, merge, skip, etc. It has been found that the motion information of previously inter-coded blocks stored in the HMVP table even not adjacent to the current block can be utilized for more efficient motion vector prediction.

After one MVP candidate is selected within the given candidate set of motion vectors for a current CU, video encoder 20 may generate one or more syntax elements for the corresponding MVP candidate and encode them into the video bitstream such that video decoder 30 can retrieve the MVP candidate from the video bitstream using the syntax elements. Depending on the specific mode used for constructing the motion vectors candidate set, different modes (e.g., AMVP, merge, skip, etc.) have different sets of syntax elements. For the AMVP mode, the syntax elements include inter prediction indicators (List 0, List 1, or bi-directional prediction), reference indices, motion vector candidate indices, motion vector prediction residual signal, etc. For the skip mode and the merge mode, only merge indices are encoded into the bitstream because the current CU inherits the other syntax elements including the inter prediction indicators, reference indices, and motion vectors from a neighboring CU referred by the coded merge index. In the case of a skip coded CU, the motion vector prediction residual signal is also omitted.

FIG. 5A is a block diagram illustrating spatially neighboring and temporally collocated block positions of a current CU to be encoded/decoded in accordance with some implementations of the present disclosure. For a given mode, a motion vector prediction (MVP) candidate list is constructed by first checking the availability of motion vectors associated with the spatially left and above neighboring block positions, and the availability of motion vectors associated with temporally collocated block positions and then the motion vectors in the HMVP table. During the process of constructing the MVP candidate list, some redundant MVP candidates are removed from the candidate list and, if necessary, zero-valued motion vector is added to make the candidate list to have a fixed length (note that different modes may have different fixed lengths). After the construction of the MVP candidate list, video encoder 20 can select the best motion vector predictor from the candidate list and encode the corresponding index indicating the chosen candidate into the video bitstream.

Using FIG. 5A as an example and assuming that the candidate list has a fixed length of two, the motion vector predictor (MVP) candidate list for the current CU may be constructed by performing the following steps in order under the AMVP mode:

- 1) Selection of MVP candidates from spatially neighboring CUs

17

- a) Derive up to one non-scaled MVP candidate from one of the two left spatial neighbour CUs starting with A0 and ending with A1;
- b) If no non-scaled MVP candidate from left is available in the previous step, derive up to one scaled MVP candidate from one of the two left spatial neighbour CUs starting with A0 and ending with A1;
- c) Derive up to one non-scaled MVP candidate from one of the three above spatial neighbour CUs starting with B0, then B1, and ending with B2;
- d) If neither A0 nor A1 is available or if they are coded in intra modes, derive up to one scaled MVP candidate from one of the three above spatial neighbour CUs starting with B0, then B1, and ending with B2;
- 2) If two MVP candidates are found in the previous steps and they are identical, remove one of the two candidates from the MVP candidate list;
- 3) Selection of MVP candidates from temporally collocated CUs
 - a) If the MVP candidate list after the previous step does not include two MVP candidates, derive up to one MVP candidate from the temporal collocated CUs (e.g., T0)
- 4) Selection of MVP candidates from the HMVP table
 - a) If the MVP candidate list after the previous step does not include two MVP candidates, derive up to two history-based MVP from the HMVP table; and
- 5) If the MVP candidate list after the previous step does not include two MVP candidates, add up to two zero-valued MVPs to the MVP candidate list.

Since there are only two candidates in the AMVP-mode MVP candidate list constructed above, an associated syntax element like a binary flag is encoded into the bitstream to indicate that which of the two MVP candidates within the candidate list is used for decoding the current CU.

In some implementations, the MVP candidate list for the current CU under the skip or merge mode may be constructed by performing a similar set of steps in order like the ones above. It is noted that one special kind of merge candidate called “pair-wise merge candidate” is also included into the MVP candidate list for the skip or merge mode. The pair-wise merge candidate is generated by averaging the MVs of the two previously derived merge-mode motion vector candidates. The size of the merge MVP candidate list (e.g., from 1 to 6) is signaled in a slice header of the current CU. For each CU in the merge mode, an index of the best merge candidate is encoded using truncated unary binarization (TU). The first bin of the merge index is coded with context and bypass coding is used for other bins.

As mentioned above, the history-based MVPs can be added to either the AMVP-mode MVP candidate list or the merge MVP candidate list after the spatial MVP and temporal MVP. The motion information of a previously inter-coded CU is stored in the HMVP table and used as a MVP candidate for the current CU. The HMVP table is maintained during the encoding/decoding process. Whenever there is a non-sub-block inter-coded CU, the associated motion vector information is added to the last entry of the HMVP table as a new candidate while the motion vector information stored in the first entry of the HMVP table is removed from therein (if the HMVP table is already full and there is no identical duplicate of the associated motion vector information in the table). Alternatively, the identical duplicate of the associated motion vector information is removed from the table before the associated motion vector information is added to the last entry of the HMVP table.

18

As noted above, intra block copy (IBC) can significantly improve the coding efficiency of screen content materials. Since IBC mode is implemented as a block-level coding mode, block matching (BM) is performed at video encoder 20 to find an optimal block vector for each CU. Here, a block vector is used to indicate the displacement from the current block to a reference block, which has already been reconstructed within the current picture. An IBC mode is treated as the third prediction mode other than the intra or inter prediction modes.

At the CU level, the IBC mode can be signaled as IBC AMVP mode or IBC skip/merge mode as follows:

IBC AMVP mode: a block vector difference (BVD) between the actual block vector of a CU and a block vector predictor of the CU selected from block vector candidates of the CU is encoded in the same way as a motion vector difference is encoded under the AMVP mode described above. The block vector prediction method uses two block vector candidates as predictors, one from left neighbor and the other one from above neighbor (if IBC coded). When either neighbor is not available, a default block vector will be used as a block vector predictor. A binary flag is signaled to indicate the block vector predictor index. The IBC AMVP candidate list consists of spatial and HMVP candidates.

IBC skip/merge mode: a merge candidate index is used to indicate which of the block vector candidates in the merge candidate list (also known as a “merge list”) from neighboring IBC coded blocks is used to predict the block vector for the current block. The IBC merge candidate list consists of spatial, HMVP, and pairwise candidates.

Another approach of improving the coding efficiency adopted by the state-of-art coding standard is to introduce the parallel processing to the video encoding/decoding process using, e.g., a multi-core processor. For example, wavefront parallel processing (WPP) has already been introduced into HEVC as a feature of encoding or decoding of multiple rows CTUs in parallel using multiple threads.

FIG. 5B is a block diagram illustrating multi-threaded encoding of multiple rows of CTUs of a picture using wavefront parallel processing (WPP) in accordance with some implementations of the present disclosure. When WPP is enabled, it is possible to process multiple rows of CTUs in parallel in a wavefront fashion, where there may be a delay of two CTUs between the start of two neighboring wavefronts. For example, to code the picture 500 using WPP, a video coder, such as video encoder 20 and video decoder 30, may divide the coding tree units (CTUs) of the picture 500 into a plurality of wavefronts, each wavefront corresponding to a respective row of CTUs in the picture. The video coder may start coding a top wavefront, e.g., using a first coder core or thread. After the video coder has coded two or more CTUs of the top wavefront, the video coder may start coding a second-to-top wavefront in parallel with coding the top wavefront, e.g., using a second, parallel coder core or thread. After the video coder has coded two or more CTUs of the second-to-top wavefront, the video coder may start coding a third-to-top wavefront in parallel with coding the higher wavefronts, e.g., using a third, parallel coder core or thread. This pattern may continue down the wavefronts in the picture 500. In the present disclosure, a set of CTUs that a video coder is concurrently coding, using WPP, is referred to as a “CTU group.” Thus, when the video coder uses WPP to code a picture, each CTU of the CTU group may belong to a unique wavefront of the picture and

the CTU may be offset from a CTU in a respective, above wavefront by at least two columns of CTUs of the picture.

The video coder may initialize a context for a current wavefront for performing context adaptive binary arithmetic coding (CABAC) of the current wavefront based on data of the first two blocks of the above wavefront, as well as one or more elements of a slice header for a slice including the first code block of the current wavefront. The video coder may perform CABAC initialization of a subsequent wavefront (or CTU row) using the context states after coding two CTUs of a CTU row above the subsequent CTU row. In other words, before beginning coding of a current wavefront, a video coder (or more specifically, a thread of the video coder) may code at least two blocks of a wavefront above the current wavefront, assuming the current wavefront is not the top row of CTUs of a picture. The video coder may then initialize a CABAC context for the current wavefront after coding at least two blocks of a wavefront above the current wavefront. In this example, each CTU row of the picture 500 is a separated partition and has an associated thread (WPP Thread 1, WPP Thread 2, . . .) such that the number of CTU rows in the picture 500 can be encoded in parallel.

Because the current implementation of the HMVP table uses a global motion vector (MV) buffer to store previously reconstructed motion vectors, this HMVP table cannot be implemented on the WPP-enabled parallel encoding scheme described above in connection with FIG. 5B. In particular, the fact that the global MV buffer is shared by all the threads of the encoding/decoding process of a video coder prevents the WPP threads after the first WPP thread (i.e., WPP Thread 1) from being started since these WPP threads have to wait for the HMVP table update from the last CTU (i.e., right-most CTU) of the first WPP thread (i.e., the first CTU row) to be completed.

To overcome the problem, it is proposed that the global MV buffer shared by the WPP threads be replaced with multiple CTU row-dedicated buffers such that each wavefront of CTU row has its own buffer for storing an HMVP table corresponding to the CTU row being processed by a corresponding WPP thread when WPP is enabled at the video coder. It is noted that each CTU row having its own HMVP table is equivalent to resetting the HMVP table before coding a first CU of the CTU row. The HMVP table reset is to flush out all the motion vectors in the HMVP table resulting from coding of another CTU row. In one implementation, the reset operation is to set the size of the available motion vector predictors in the HMVP table to be zero. In yet another implementation, the reset operations could be to set the reference index of all the entries in the HMVP table to be an invalid value such as -1. By doing so, the construction of MVP candidate list for a current CTU within a particular wavefront, regardless of which one of the three modes, AMVP, merge, and skip, is dependent upon an HMVP table associated with a WPP thread processing the particular wavefront. There is no inter-dependency between different wavefronts other than the two-CTU delay described above and the construction of motion vector candidate lists associated with different wavefronts can proceed in parallel like the WPP process depicted in FIG. 5B. In other words, at the beginning of processing a particular wavefront, the HMVP table is reset to be empty without affecting the coding of another wavefront of CTUs by another WPP thread. In some cases, the HMVP table can be reset to be empty before the coding of each individual CTU. In this case, the motion vectors in the HMVP table are limited to a particular CTU and there is probably a higher

chance of a motion vector within the HMVP table being selected as a motion vector of a current CU within the particular CTU.

FIGS. 6A and 6B are block diagrams illustrating an exemplary reconstructed luma block 602 and an exemplary associated chroma block 620, respectively, in accordance with some implementations of the present disclosure. In this example, luma samples of reconstructed luma block 602 (e.g., luma sample 604), top neighboring luma group 606 (e.g., luma sample 608), and left neighboring luma group 610 (e.g., luma sample 613) have been predicted during a video coding process. Chroma samples of chroma block 620 are to be predicted, while chroma samples of top neighboring chroma group 624 (e.g., chroma sample 626) and left neighboring chroma group 628 (e.g., chroma sample 630) have been predicted during the video coding process. In some embodiments, chroma samples of chroma block 620 can be predicted by applying a cross component linear model (CCLM) to the corresponding down-sampled luma samples of reconstructed luma block 602. Derivation and application of the CCLM is provided below in connection with FIGS. 7A-7E.

In some embodiments, reconstructed luma block 602 and chroma block 620 each represent a different component of a portion of a reconstructed video frame. For example, in the YCbCr color space, an image is represented by a luma component (Y), a blue-difference chroma component (Cb), and a red-difference chroma component (Cr). Reconstructed luma block 602 represents the luma component (i.e., brightness) of a portion of the video frame, and chroma block 620 represents a chroma component (i.e., color) of the same portion of the video frame. A luma sample (e.g., luma sample 604) of reconstructed luma block 602 has a luma value representing the brightness at a particular pixel of the video frame, and a chroma sample (e.g., chroma sample 622) has a chroma value representing the color at a particular pixel of the video frame.

In some embodiments, reconstructed luma block 602 is a $2M \times 2N$ block with $2M$ luma samples across the block width and $2N$ luma samples across the block height. M and N can be the same value (e.g., reconstructed luma block 602 is a square block) or different values (e.g., reconstructed luma block 602 is a non-square block).

Chroma subsampling is a common compression technique as human visual system is less sensitive to the color difference than to brightness difference. As a result, reconstructed luma block 602 and chroma block 620 may represent the same portion of a video frame but are encoded with different resolutions. For example, the video frame may have been encoded using a chroma subsampling scheme (e.g., 4:2:0 or 4:2:2) to encode for chroma information than for luma information with less resolution. As illustrated in FIGS. 6A and 6B, reconstructed luma block 602 is encoded with a resolution of $2M \times 2N$, while chroma block 620 is encoded with a smaller resolution of $M \times N$. In practice, chroma block 620 can have other resolution such as $2M \times 2N$ (e.g., 4:4:4 full sampling), $2M \times N$ (e.g., 4:4:0 sub-sampling), $M \times 2N$ (e.g., 4:2:2 sub-sampling), and $\frac{1}{2}M \times 2N$ (e.g., 4:1:1 sub-sampling).

Reconstructed luma block 602 is next to top neighboring luma group 606 and left neighboring luma group 610. The size of the top neighboring luma group and left neighboring luma group can be explicitly signaled or dependent upon the size of reconstructed luma block 602. For example, top neighboring luma group 606 can have a width of $2M$ samples (e.g., same as the width of reconstructed luma block 602) or $4M$ samples (e.g., double that of the width of

21

reconstructed luma block **602**), and a height of 2 samples. Left neighboring luma group **610** can have a width of 2 samples, with a height of 2N or 4N samples. In some embodiments, top neighboring luma group **606** and left neighboring luma group **610** are each a portion of another luma block or blocks of the same video frame that have been reconstructed.

Chroma block **620** is next to top neighboring chroma group **624** and left neighboring group **628**. The size of top neighboring chroma group **624** and left neighboring group **628** can be explicitly signaled or dependent upon the size of chroma block **620**. For example, top neighboring chroma group **624** can have a size of $1 \times M$, and left neighboring chroma group **628** can have a size of $N \times 1$.

In some embodiments, chroma values (e.g., chroma values for chroma block **620**) can be predicted based on luma values of reconstructed luma samples (e.g., luma sample **604**). For example, under the assumption that there exists a linear or quasi-linear relation between luma values and corresponding chroma values of a video frame, a video codec can predict chroma values based on corresponding reconstructed luma values using the CCLM. By doing so, the video codec can save a significant amount of time and bandwidth for encoding the chroma values, transmitting the encoded chroma values, and decoding the encoded chroma values. To use the CCLM to predict chroma samples from luma samples, the video codec (1) derives a linear model between the chroma samples and the luma samples, and (2) applies the linear model to reconstructed luma samples that correspond to the chroma samples to be predicted.

In some embodiments, since luma blocks and chroma blocks are of different resolutions (e.g., the chroma blocks having been sub-sampled), the video codec first performs down-sampling on luma samples to generate down-sampled luma samples (e.g., down-sampled luma sample **605**, **609**, and **612**) that uniquely correspond to respective chroma samples. In some embodiments, six neighboring reconstructed luma samples in both height and width direction of the video frame are used to generate a down-sampled luma sample (e.g., weighted-averaging schemes known in the art including six-tap down-sampling or like). For example, the six reconstructed luma samples within region **611** (each represented by a small box in the figure) inside top neighboring luma group are used to generate down-sampled luma sample **609** through averaging of their corresponding luma values, and the six reconstructed luma samples within region **607** (each represented by a small box in the figure) inside reconstructed luma block **602** are used to generate down-sampled luma sample **605**. Alternatively, a down-sampled luma sample is generated by identifying a reconstructed luma sample in a region of interest, or using a different number of reconstructed luma samples in a region of a different shape.

In some embodiments, to drive the linear model, the video codec uses a Max-Min method by identifying maximum and minimum of down-sampled luma samples (e.g., the down-sampled luma samples having the maximum and the minimum luma values, respectively) and the corresponding reconstructed chroma samples, and fitting a linear model (e.g., $Y = \alpha X + \beta$) through the maximum and minimum data points (e.g., the maximum data point includes the down-sampled luma sample having the maximum luma value and the corresponding reconstructed chroma sample, and the minimum data point includes the down-sampled luma sample having the minimum luma value and the corresponding reconstructed chroma sample). After the linear model is derived, the video codec applies the linear model to down-

22

sampled luma samples in reconstructed luma block **602** to generate corresponding chroma samples of chroma block **620**. The video codec can obtain the max and min data points in the following way:

1. In some embodiments, the video codec searches a group of down-sampled luma samples (e.g., a selected group of down-sampled luma samples in top neighboring luma group **606** and left neighboring luma group **610**) to identify the maximum down-sampled luma sample and the minimum down-sampled luma sample. The video codec then identifies previously-reconstructed chroma samples (e.g., reconstructed chroma samples in top neighboring chroma group **624** and left neighboring chroma group **628**) corresponding to the maximum and minimum down-sampled luma samples, as described below in connection with FIG. 7A and the related description for detail on this implementation.

2. In some embodiments, the video codec searches a group of reconstructed luma samples (e.g., a selected group of reconstructed luma samples in top neighboring luma group **606** and left neighboring luma group **610**) to identify (i) a reconstructed luma sample having the maximum luma value and (ii) a reconstructed luma sample having the minimum luma value in the selected group of reconstructed luma samples without performing down-sampling on the selected group of reconstructed luma samples to identify the maximum and minimum reconstructed luma samples. The video codec then performs down-sampling in a region (e.g., a region with six samples using weighted-averaging schemes known in the art including six-tap down-sampling or like) associated with the maximum and the minimum reconstructed luma samples to generate a down-sampled luma sample as the maximum reconstructed luma sample (which may or may not be exactly the maximum down-sampled luma sample) and a down-sampled luma sample as the min reconstructed luma sample (which may or may not be exactly the minimum down-sampled luma sample). The video codec then identifies a reconstructed chroma sample (e.g., in top neighboring chroma group **624** and left neighboring chroma group **628**) corresponding to the down-sampled luma sample identified as the maximum reconstructed luma sample, and a reconstructed chroma sample corresponding to the down-sampled luma sample identified as the minimum reconstructed luma sample, as described below in connection with FIG. 7B and the related description for detail on this implementation.

3. In some embodiments, the video codec searches a group of reconstructed chroma samples (e.g., selected chroma samples in top neighboring chroma group **624** and left neighboring chroma group **628**) to identify the maximum and the minimum reconstructed chroma samples (e.g., the chroma samples having the maximum and minimum chroma values, respectively). The video codec then identifies the down-sampled luma samples (e.g., down-sampled luma samples in top neighboring luma group **606** and left neighboring luma group **610**) corresponding to the maximum and min reconstructed chroma samples, as describe below in connection with FIG. 7C and the related description for detail on this implementation.

4. In some embodiments, the video codec searches a group of down-sampled luma samples (e.g., a selected group of down-sampled luma samples in top neighboring luma group **606** and left neighboring luma group **610**) to identify a predefined number (e.g., two) of down-sampled luma samples having the largest luma values) and a predefined number (e.g., two) of down-sampled luma samples having the smallest luma values. The video codec then identifies reconstructed chroma samples in top neighboring chroma

group **624** and left neighboring chroma group **628**, respectively, each corresponding to a respective one of the group of maximum down-sampled luma samples and the group of minimum down-sampled luma samples, respectively. The video codec then performs a weighted average of values (e.g., chroma or luma values) within each of the identified groups of reconstructed chroma and down-sampled luma samples to generate a maximum averaged chroma value, a minimum averaged chroma value, a maximum averaged down-sampled luma value (e.g., from the group of maximum down-sampled luma samples), and a minimum averaged down-sampled luma value (e.g., from the group of minimum down-sampled luma samples), as described below in connection with FIGS. 7D and 7E and the related description for detail on this implementation.

FIGS. 7A-7E illustrate various approaches of using the CCLM to derive a linear model between luma values and chroma values in accordance with some implementations of the present disclosure. In particular, each circle data point (e.g., point **702a**) on a plot represents a pair of reconstructed chroma sample in the horizontal axis and a corresponding reconstructed luma sample in the vertical axis. For example, a reconstructed chroma sample corresponds to a reconstructed luma sample if a down-sampled luma sample generated in part using the reconstructed luma sample (e.g., using weighted-averaging schemes known in the art including six-tap down-sampling or like) corresponds to the reconstructed chroma sample. Each square data point (e.g., point **702b**) on the plot represents a pair of reconstructed chroma sample in the horizontal axis and a corresponding down-sampled luma sample in the vertical axis. In some embodiments, since multiple reconstructed luma samples are used to generate a down-sampled luma sample (e.g., using the weighted-averaging schemes known in the art including six-tap down-sampling or like), a square data point is associated with multiple circle data points. The dotted rectangle (e.g., the dotted rectangle **703**) indicates that the encompassed square data point and the circle data points are associated (e.g., the down-sampled luma sample corresponding to the square data point is generated from the reconstructed luma samples corresponding to the circle data points). For illustrative purpose, only one square data point and one circle data point are shown inside a dotted rectangle, although in actual practice, each dotted rectangle can include multiple circle data points and one square data point.

In some embodiments, the video codec searches the down-sampled luma samples (e.g., down-sampled luma samples **609** and **612** of FIG. 6A) to identify down-sampled luma samples having the maximum and minimum luma values and corresponding reconstructed chroma samples to derive the linear model. For example, in FIG. 7A, square data point **702b** represents a down-sampled luma sample having the minimum luma value of the selected group of down-sampled luma samples and a corresponding reconstructed chroma sample, and square data point **704b** represents a down-sampled luma sample having the maximum luma value and a corresponding reconstructed chroma sample. As a result, a line fitting through point **702b** and point **704b** represents the linear model to predict chroma samples (e.g., chroma sample **622** of chroma block **620** in FIG. 6B) from down-sampled luma samples (e.g., down-sampled luma sample **605** of luma block **602** in FIG. 6A).

Generating the down-sampled luma samples is computationally intensive as most of the down-sampled luma samples are used in a very little manner in the Max-Min method. In some embodiments, instead of performing down-sampling of all the luma samples, the video codec directly

searches reconstructed luma samples (e.g., reconstructed luma sample **608** and **613** of FIG. 6A) to identify reconstructed luma samples having the maximum and minimum luma values. As illustrated in FIG. 7B, circle data point **702a** represents a reconstructed luma sample having the minimum luma value and a corresponding reconstructed chroma sample, and circle data point **706a** represents a reconstructed luma sample having the maximum luma value and a corresponding reconstructed chroma sample. After identifying the minimum and maximum reconstructed luma samples (circle data point **702a** and **706a**), the video codec then performs down-sampling in a region including the minimum and maximum reconstructed luma samples (e.g., using weighted-averaging schemes known in the art including six-tap down-sampling or like) to generate corresponding quasi minimum and maximum down-sampled luma samples (represented by square data points **702b** and **706b** in FIG. 7B, which may or may not be the same as square data points **702b** and **704b** in FIG. 7A). A line fitting through square data points **702b** and **706b** in FIG. 7B represents the linear model to predict chroma samples from reconstructed luma samples. Compared to the method used in FIG. 7A, only two down-sampling operations are performed. The identified min down-sampled luma sample is the same as the one used in FIG. 7A, while the max down-sampled luma sample is different from the one used in FIG. 7A.

In some embodiments, the video codec generates the linear model using the reconstructed luma samples with the maximum and minimum luma values, and forgoes performing down-sampling. In FIG. 7C, no down-sampling is performed on reconstructed luma samples, and a linear model is generated by fitting a line through circle data point **702a** and **706a** directly.

In some embodiments, instead of searching for down-sampled luma samples (or reconstructed luma samples) with maximum and minimum luma values, the video codec first searches for reconstructed chroma samples having the maximum and minimum chroma values. Once the maximum and minimum reconstructed chroma samples are identified, the video codec then computes the corresponding down-sampled luma samples to generate the linear model. In FIG. 7D, circle data point **708a** represents the reconstructed chroma sample having the minimum chroma value, and circle data point **704a** represents the reconstructed chroma sample having the maximum chroma value. The video codec then generates a linear model fitting through square data point **708b** (which represents a down-sampled luma sample generated in part using the reconstructed luma sample from circle data point **708a**) and square data point **704b** (which represents a down-sampled luma sample generated in part using the reconstructed luma sample from circle data point **704a**).

In some embodiments, the video codec selects a number (e.g., two) of down-sampled luma samples having the largest luma values, and a number (e.g., two) of down-sampled luma samples having the smallest luma values. The video codec then searches for the corresponding maximum and minimum groups of reconstructed chroma samples. The video codec performs an averaging operation within each group, and uses the averaged luma and chroma values to generate a linear model. In FIG. 7E, the largest two down-sampled luma samples (square data points **710b** and **704b**) and the smallest two down-sampled luma samples (square data points **708b** and square data points **702b**) are used to generate the linear model.

FIG. 8 is a flowchart illustrating an exemplary process **700** by which a video codec implements the techniques of

using a cross-component linear model to reconstruct chroma samples for a chroma block based on reconstructed luma samples from a luma block. Process 700 can be implemented during either a decoding process or an encoding process.

As the first step, the video codec reconstructs a luma block corresponding to a chroma block (810). The chroma block (e.g., chroma block 620 of FIG. 6B) is to be subsequently reconstructed from the reconstructed luma block and may be of a different resolution different from that of the reconstructed luma block (e.g., reconstructed luma block 602 of FIG. 6A). The luma block corresponds to the chroma block as they represent different components (e.g., brightness component and color component, respectively) of the same portion of a video frame. In some embodiments, the luma block is adjacent to a plurality of previously-reconstructed neighboring luma samples (e.g., reconstructed luma samples in top neighboring luma group 606 and left neighboring luma group 610 of FIG. 6A), and the chroma block is adjacent to a plurality of previously-reconstructed neighboring chroma samples (e.g., reconstructed chroma samples in top neighboring chroma group 624 and left neighboring chroma group 628 of FIG. 6A). Note that the term "adjacent to" in this application is not limited to "immediately next to" and also covers the situation in which the coding block is not immediately next to the luma/chroma samples. In some embodiments, the video codec predicts chroma samples in the chroma block by deriving a cross-component linear model and applies the model to reconstructed luma samples (or down-sampled reconstructed luma samples in the luma block).

Next, the video codec searches a sub-group of the plurality of reconstructed neighboring luma samples in a pre-defined order to identify at least one maximum or quasi-maximum luma sample and at least one minimum or quasi-minimum luma sample (820). In some embodiments, the sub-group of the plurality of reconstructed neighboring luma samples covers all the plurality of reconstructed neighboring luma samples. For example, the video codec can search all luma samples in the neighboring luma samples or up to a predefined number of neighboring luma samples. In some embodiments, the video codec searches the neighboring luma samples according to the raster scan order, from left to right, from top to down, or in any combination of these orders. In some embodiments, the neighboring luma samples include those samples spatially on top of the luma block (e.g., top neighboring chroma group 624) and those samples to the left of the luma block (e.g., left neighboring chroma group 610). The video codec may search only the top neighboring chroma samples or the left neighboring chroma samples.

After identifying the maximum and minimum luma samples, the video codec computes at least one down-sampled maximum luma sample and at least one down-sampled minimum luma sample corresponding to the identified maximum and minimum luma samples, respectively (830 and 840). For example, the video codec may use the six-tap down-sampling technique to generate a down-sampled luma sample (e.g., by weighted-averaging) from six neighboring reconstructed luma samples (e.g., arranged in a 3x2 manner or a 2x3 manner). The down-sampled maximum luma sample and the down-sampled minimum luma sample each correspond to a respective reconstructed chroma sample (e.g., first reconstructed neighboring chroma sample and second reconstructed neighboring chroma sample, respectively). For example, the first reconstructed neighboring chroma sample and the second reconstructed

neighboring chroma sample can be of the plurality of reconstructed neighboring chroma samples.

In some implementations, the operations 820, 830, and 840 described above are re-arranged in a different order. For example, the video codec first computes one or more down-sampled luma samples from the plurality of reconstructed neighboring luma samples and then searches, among a sub-group of the one or more computed down-sampled luma samples, to identify at least one down-sampled maximum luma sample and at least one down-sampled minimum luma sample, respectively. The down-sampled maximum luma sample is chosen to correspond to the first reconstructed chroma sample and the down-sampled minimum luma sample is chosen to correspond to the second reconstructed chroma sample.

Next, the video codec generates a linear model using a first pair of the down-sampled maximum luma sample and the first reconstructed neighboring chroma sample and a second pair of the down-sampled minimum luma sample and the second reconstructed neighboring chroma sample (850). In some embodiments, the video codec generates the linear model using the max-min method by identifying two data points (e.g., (first reconstructed neighboring chroma sample, down-sampled maximum luma sample) and (second reconstructed neighboring chroma sample, down-sampled minimum luma sample)) and fits a linear equation through the two data points.

After obtaining the linear model, the video codec computes down-sampled luma samples from luma samples of the reconstructed luma block (860). Each down-sampled luma sample corresponds to a chroma sample of the chroma block. For example, the video codec can compute down-sampled luma samples using the same down-sampling technique used to compute the down-sampled max and min luma samples.

Finally, the video codec predicts chroma samples in the chroma block by applying the linear model to the corresponding down-sampled luma samples (870).

In one or more examples, the functions described may be implemented in hardware, software, firmware, or any combination thereof. If implemented in software, the functions may be stored on or transmitted over, as one or more instructions or code, a computer-readable medium and executed by a hardware-based processing unit. Computer-readable media may include computer-readable storage media, which corresponds to a tangible medium such as data storage media, or communication media including any medium that facilitates transfer of a computer program from one place to another, e.g., according to a communication protocol. In this manner, computer-readable media generally may correspond to (1) tangible computer-readable storage media which is non-transitory or (2) a communication medium such as a signal or carrier wave. Data storage media may be any available media that can be accessed by one or more computers or one or more processors to retrieve instructions, code and/or data structures for implementation of the implementations described in the present application. A computer program product may include a computer-readable medium.

The terminology used in the description of the implementations herein is for the purpose of describing particular implementations only and is not intended to limit the scope of claims. As used in the description of the implementations and the appended claims, the singular forms "a," "an," and "the" are intended to include the plural forms as well, unless the context clearly indicates otherwise. It will also be understood that the term "and/or" as used herein refers to

27

and encompasses any and all possible combinations of one or more of the associated listed items. It will be further understood that the terms “comprises” and/or “comprising,” when used in this specification, specify the presence of stated features, elements, and/or components, but do not preclude the presence or addition of one or more other features, elements, components, and/or groups thereof.

It will also be understood that, although the terms first, second, etc. may be used herein to describe various elements, these elements should not be limited by these terms. These terms are only used to distinguish one element from another. For example, a first electrode could be termed a second electrode, and, similarly, a second electrode could be termed a first electrode, without departing from the scope of the implementations. The first electrode and the second electrode are both electrodes, but they are not the same electrode.

The description of the present application has been presented for purposes of illustration and description, and is not intended to be exhaustive or limited to the invention in the form disclosed. Many modifications, variations, and alternative implementations will be apparent to those of ordinary skill in the art having the benefit of the teachings presented in the foregoing descriptions and the associated drawings. The embodiment was chosen and described in order to best explain the principles of the invention, the practical application, and to enable others skilled in the art to understand the invention for various implementations and to best utilize the underlying principles and various implementations with various modifications as are suited to the particular use contemplated. Therefore, it is to be understood that the scope of claims is not to be limited to the specific examples of the implementations disclosed and that modifications and other implementations are intended to be included within the scope of the appended claims.

What is claimed is:

1. A method for decoding a video signal, comprising:
 - reconstructing a luma block corresponding to a chroma block, wherein the luma block is adjacent to a plurality of reconstructed neighboring luma samples, and wherein the chroma block is adjacent to a plurality of reconstructed neighboring chroma samples;
 - computing a plurality of down-sampled luma samples from the plurality of reconstructed neighboring luma samples;
 - identifying, from a sub-group of the plurality of computed down-sampled luma samples, two down-sampled maximum luma samples, wherein the two down-sampled maximum luma samples correspond to two first reconstructed chroma samples of the plurality of reconstructed neighboring chroma samples respectively, and the sub-group is composed of a predefined number of the computed down-sampled luma samples among the plurality of computed down-sampled luma samples;
 - identifying, from the sub-group of the plurality of computed down-sampled luma samples, two down-sampled minimum luma samples, wherein the two down-sampled minimum luma samples correspond to two second reconstructed chroma samples of the plurality of reconstructed neighboring chroma samples respectively;
 - averaging the two down-sampled maximum luma samples, the two down-sampled minimum luma samples, the two first reconstructed chroma samples, and the two second reconstructed chroma samples, respectively, to obtain an averaged down-sampled

28

- maximum luma sample, an averaged down-sampled minimum luma sample, an averaged first reconstructed chroma sample and an averaged second reconstructed chroma sample;
 - fitting a linear model through the averaged down-sampled maximum luma sample, the averaged down-sampled minimum luma sample, the averaged first reconstructed chroma sample, and the averaged second reconstructed chroma sample;
 - computing down-sampled luma samples from luma samples of the reconstructed luma block, wherein each down-sampled luma sample corresponds to a chroma sample of the chroma block; and
 - predicting chroma samples of the chroma block by applying the linear model to the corresponding computed down-sampled luma samples.
2. The method of claim 1, wherein the chroma block and the luma block are encoded using a 4:2:0 or 4:2:2 chroma sub-sampling scheme, and wherein the chroma block and the luma block have different resolutions.
 3. The method of claim 1, wherein the plurality of reconstructed neighboring luma samples includes luma samples located above the reconstructed luma block and/or luma samples to left of the reconstructed luma block.
 4. The method of claim 1, wherein the computing down-sampled luma samples from luma samples of the reconstructed luma block comprises performing a weighted average of six neighboring luma samples to the luma sample.
 5. The method of claim 1, wherein the fitting the linear model comprises fitting a linear equation through a data point associated with the averaged down-sampled maximum luma sample and the averaged first reconstructed chroma sample and a data point associated with the averaged down-sampled minimum luma sample and the averaged second reconstructed chroma sample.
 6. A decoding device comprising:
 - one or more processors;
 - memory coupled to the one or more processors; and
 - a plurality of programs stored in the memory that, when executed by the one or more processors, cause the one or more processors to perform acts comprising:
 - reconstructing a luma block corresponding to a chroma block, wherein the luma block is adjacent to a plurality of reconstructed neighboring luma samples, and wherein the chroma block is adjacent to a plurality of reconstructed neighboring chroma samples;
 - computing a plurality of down-sampled luma samples from the plurality of reconstructed neighboring luma samples;
 - identifying, from a sub-group of the plurality of computed down-sampled luma samples, two down-sampled maximum luma samples, wherein the two down-sampled maximum luma samples correspond to two first reconstructed chroma samples of the plurality of reconstructed neighboring chroma samples respectively, and the sub-group is composed of a predefined number of the computed down-sampled luma samples among the plurality of computed down-sampled luma samples;
 - identifying, from the sub-group of the plurality of computed down-sampled luma samples, two down-sampled minimum luma samples, wherein the two down-sampled minimum luma samples correspond to two second reconstructed chroma samples of the plurality of reconstructed neighboring chroma samples respectively;

29

averaging the two down-sampled maximum luma samples, the two down-sampled minimum luma samples, the two first reconstructed chroma samples, and the two second reconstructed chroma samples, respectively, to obtain an averaged down-sampled maximum luma sample, an averaged down-sampled minimum luma sample, an averaged first reconstructed chroma sample and an averaged second reconstructed chroma sample;

fitting a linear model through the averaged down-sampled maximum luma sample, the averaged down-sampled minimum luma sample, the averaged first reconstructed chroma sample, and the averaged second reconstructed chroma sample;

computing down-sampled luma samples from luma samples of the reconstructed luma block, wherein each down-sampled luma sample corresponds to a chroma sample of the chroma block; and

predicting chroma samples of the chroma block by applying the linear model to the corresponding computed down-sampled luma samples.

7. The decoding device of claim 6, wherein the chroma block and the luma block are encoded using a 4:2:0 or 4:2:2 chroma sub-sampling scheme, and wherein the chroma block and the luma block have different resolutions.

8. The decoding device of claim 6, wherein the plurality of reconstructed neighboring luma samples includes luma samples located above the reconstructed luma block and/or luma samples to left of the reconstructed luma block.

9. The decoding device of claim 6, wherein the computing down-sampled luma samples from luma samples of the reconstructed luma block comprises performing a weighted average of six neighboring luma samples to the luma sample.

10. The decoding device of claim 6, wherein the fitting the linear model comprises fitting a linear equation through a data point associated with the averaged down-sampled maximum luma sample and the averaged first reconstructed chroma sample and a data point associated with the averaged down-sampled minimum luma sample and the averaged second reconstructed chroma sample.

11. A method for storing a bitstream, comprising:
performing an encoding method to generate a bitstream;
and

storing the bitstream,

wherein the encoding method comprises:

reconstructing a luma block corresponding to a chroma block, wherein the luma block is adjacent to a plurality of reconstructed neighboring luma samples, and wherein the chroma block is adjacent to a plurality of reconstructed neighboring chroma samples;

computing a plurality of down-sampled luma samples from the plurality of reconstructed neighboring luma samples;

identifying, from a sub-group of the plurality of computed down-sampled luma samples, two down-sampled maximum luma samples, wherein the two down-sampled maximum luma samples correspond to two

30

first reconstructed chroma samples of the plurality of reconstructed neighboring chroma samples respectively, and the sub-group is composed of a predefined number of the computed down-sampled luma samples among the plurality of computed down-sampled luma samples;

identifying, from the sub-group of the plurality of computed down-sampled luma samples, two down-sampled minimum luma samples, wherein the two down-sampled minimum luma samples correspond to two second reconstructed chroma samples of the plurality of reconstructed neighboring chroma samples respectively;

averaging the two down-sampled maximum luma samples, the two down-sampled minimum luma samples, the two first reconstructed chroma samples, and the two second reconstructed chroma samples, respectively, to obtain an averaged down-sampled maximum luma sample, an averaged down-sampled minimum luma sample, an averaged first reconstructed chroma sample and an averaged second reconstructed chroma sample;

fitting a linear model through the averaged down-sampled maximum luma sample, the averaged down-sampled minimum luma sample, the averaged first reconstructed chroma sample, and the averaged second reconstructed chroma sample;

computing down-sampled luma samples from luma samples of the reconstructed luma block, wherein each down-sampled luma sample corresponds to a chroma sample of the chroma block; and

predicting chroma samples of the chroma block by applying the linear model to the corresponding computed down-sampled luma samples,

wherein the bitstream is to be decoded by the method of claim 1.

12. The method of claim 11, wherein the chroma block and the luma block are encoded using a 4:2:0 or 4:2:2 chroma sub-sampling scheme, and wherein the chroma block and the luma block have different resolutions.

13. The method of claim 11, wherein the plurality of reconstructed neighboring luma samples includes luma samples located above the reconstructed luma block and/or luma samples to left of the reconstructed luma block.

14. The method of claim 11, wherein the computing down-sampled luma samples from luma samples of the reconstructed luma block comprises performing a weighted average of six neighboring luma samples to the luma sample.

15. The method of claim 11, wherein the fitting the linear model comprises fitting a linear equation through a data point associated with the averaged down-sampled maximum luma sample and the averaged first reconstructed chroma sample and a data point associated with the averaged down-sampled minimum luma sample and the averaged second reconstructed chroma sample.

* * * * *